

Universidad de La Habana
Facultad de Matemática y Computación



DermaUH versión 3.0: Integración de nuevos modelos para procesamiento de imágenes

Autor:

José Miguel Leyva De la Cruz

Tutores:

Dra. Marta L. Barguer Díaz-Romañach

Lic. Joanna Campbell Amos

Lic. Alejandro Beltrán

Trabajo de Diploma
presentado en opción al título de
Licenciado en Ciencia de la Computación

Mayo 2026

github.com/josemiley02/DermaUH3.0-Application

A mi pareja, mi familia y mis amigos, que hicieron esto posible.

Agradecimientos

Quiero agradecer primero a la mujer más fuerte que he conocido en mi vida, mi consejera, mi primer pilar, el hombro donde siempre podré apoyarme, a la que le debo la vida, la sabiduría y a la que más enseñanzas me ha transmitido, quiero darle las gracias a mi madre. Agradecer también al resto de mi familia, mi padre, mi abuela, mi hermanita sin ellos yo no estaría aquí hoy, su apoyo incondicional me mantuvo firme frente a las adversidades y sirvió de impulso para hoy poder presentar esta tesis. También me gustaría agradecer a mi abuelo, que sin él, yo no tendría laptop para poder afrontar esta carrera tan difícil y rigurosa, pero que me ha traído muy buenos recuerdos.

De la facultad quiero agradecer a todo el colectivo de profesores que me impartieron clases e hicieron posible mi formación académica. Dar gracias también a mi tutora Marta Lourdes que confió en mi y me acogió como uno de sus pupilos en la recta final de mi vida como estudiante. Como no mencionar a mis amigos de la facultad, ese famoso grupo Team Five, fueron los primeros con los que me junté, con los que estudiaba y realizaba los proyectos. No puedo olvidarme tampoco de mis amistades del “Tiburón”, que me acompañaron toda la carrera y con los que luché codo a codo por salir adelante.

Lo mejor siempre se deja para el final. Quiero agradecer a mi compañera de vida, mi pareja, mi amante, mi mejor amiga, a mi Samantha, “La Samy”, quien estuvo conmigo en todo el proceso de construcción de la tesis, apoyándome y dándome ánimos cuando me venía abajo. Como olvidar esas madrugadas en Baracoa escribiendo proyectos de la escuela, trabajando o redactando capítulos de la tesis. Como olvidar como me presionaba, en el buen sentido, para que todo saliera bien. Estoy muy agradecido con ella por todo lo que me aporta, todo lo que me suma. A ella, por la que siento un amor incondicional, le agradezco.

Opinión del tutor

Opiniones de los tutores

Resumen

El presente trabajo de diploma presenta la propuesta de aplicación web DermaUH 3.0, desarrollada con el objetivo de suplir diversas dificultades de la versión anterior e incluir mejoras tanto de rendimiento como en sus modelos de clasificación de imágenes.

Para el proceso de investigación se analizaron las diferentes herramientas existentes en el mercado que utilizan inteligencia artificial para los diagnósticos asistidos, atendiendo a sus ventajas y desventajas para los doctores cubanos, y se tomaron ideas para el desarrollo de la interfaz de usuario. Posteriormente se revisaron diferentes sitios académicos especializados en la colección de imágenes dermatoscópicas, argumentando por qué estos no podían ser utilizados para entrenar los nuevos modelos de aprendizaje de máquinas. Finalmente se realizó un estudio de diferentes usos de los modelos explicativos de inteligencia artificial (XAI) para obtener experiencia sobre cómo integrarlos a una aplicación web.

A raíz de la investigación descrita anteriormente, se llegó a una propuesta de solución que cubre las necesidades planteadas para la nueva versión de DermaUH haciendo uso de Django, React y PostgreSQL como tecnologías principales. Se siguió el patrón de Arquitectura por Capas para el desarrollo de la aplicación, donde cada nivel presenta su propia propuesta de diseño. Para el frontend se utilizó una arquitectura basada en características y para el backend se implementaron varias aplicaciones Django almacenadas en un mismo servidor.

El entrenamiento de los nuevos modelos se realizó con un conjunto de imágenes de pieles cubanas proporcionado por las autoridades del ministerio de Salud Pública de Cuba (MINSAP). Esta acción fue llevada a cabo con la herramienta Kaggle para ganar en tiempo de ejecución debido al potente procesador que brinda dicha herramienta.

Finalizado el proceso de desarrollo, el principal resultado fue la obtención de una aplicación web capaz de brindar diagnóstico asistido por Inteligencia Artificial. El modelo utilizado fue entrenado con pieles cubanas y se vinculó con un modelo explicativo para justificar los diagnósticos brindados.

Abstract

This diploma thesis presents the proposed web application DermaUH 3.0, developed to address various shortcomings of the previous version and include improvements in both performance and image classification models.

For the research process, the different existing tools on the market that use artificial intelligence for computer-assisted diagnoses were analyzed, considering their advantages and disadvantages for Cuban physicians, and ideas were gathered for the development of the user interface. Subsequently, various academic websites specializing in the collection of dermatoscopic images were reviewed, and arguments were presented as to why these could not be used to train the new machine learning models. Finally, a study of different uses of explanatory models of artificial intelligence (XAI) was conducted to gain experience on how to integrate them into a web application.

As a result of the research described above, a proposed solution was developed that addresses the needs identified for the new version of DermaUH, using Django, React, and PostgreSQL as the main technologies. The Layered Architecture pattern was followed for the application's development, where each layer presents its own design proposal. A feature-based architecture was used for the frontend, and several Django applications stored on the same server were implemented for the backend.

The training of the new models was performed using a set of Cuban skin images provided by the Cuban Ministry of Public Health (MINSAP). This was done using the Kaggle tool to improve execution time due to its powerful processor capabilities.

Upon completion of the development process, the main result was a web application capable of providing AI-assisted diagnosis. The model used was trained with Cuban skins and was linked to an explanatory model to justify the diagnoses provided.

Índice general

1. Introducción	1
1.1. Motivación	2
1.2. Antecedentes	2
1.3. Formulación del problema	3
1.4. Objetivos Generales y Específicos	4
1.5. Justificación	5
1.6. Estructura del Trabajo	5
2. Estado del Arte	7
2.1. Herramientas existentes	7
2.1.1. DermEngine	8
2.1.2. SkinVision	10
2.1.3. VisualDx	12
2.1.4. Conclusiones del análisis de herramientas	13
2.2. Archivos académicos de referencia	14
2.2.1. ISIC Archive: Repositorio académico de referencia	14
2.2.2. DERM12345	17
2.2.3. HAM10000	17
2.2.4. Conclusiones parciales sobre los archivos académicos de referencia	17
2.3. Técnicas de inteligencia artificial explicable XAI en dermatología	18
3. Metodología y Análisis del Sistema Propuesto	19
3.1. Metodología	19
3.2. Levantamiento de Requerimientos	21
3.2.1. Requerimientos Funcionales	22
3.2.2. Requerimientos No Funcionales	23
3.3. Análisis y Diseño	23
3.3.1. Diagrama de Casos de Uso	23
3.3.2. Diagramas de Secuencia	27
3.3.3. Análisis y propuesta arquitectónica	28

3.3.4. Modelado de la Base de Datos	40
4. Detalles de Implementación	44
4.1. Stack Tecnológico	44
4.2. Implementaciones	46
4.2.1. Backend	46
4.2.2. Frontend	54
4.3. Modelos Machine Learning	59
5. Pruebas Funcionales	61
5.1. Pruebas de Flujo Administrador	61
5.1.1. Solicitud de Registro	61
5.2. Pruebas de Flujo Doctor	64
5.2.1. CRUD sobre Paciente	64
Conclusiones	65
Recomendaciones	66
Bibliografía	67

Índice de figuras

2.1. Logo de DermEngine	8
2.2. Logo de SkinVision	10
2.3. Estudio clínico de SkinVision	10
2.4. Diagrama del algoritmo de SkinVision	11
2.5. Logo de VisualDx	12
3.1. Diagrama UML del actor Admin	25
3.2. Diagrama UML del actor Doctor	26
3.3. Diagrama de Secuencia del actor Admin	28
3.4. Diagrama de Secuencia del actor Doctor	29
3.5. Esquema de la Arquitectura Hexagonal (Puertos y Adaptadores). Se observa el dominio en el centro, los puertos en los lados del hexágono y los adaptadores conectados a ellos. Imagen adaptada de Cockburn[19].	34
3.6. Esquema Onion Architecture, Tomado de Palermo (2008)	35
3.7. Esquema Clean Architecture en forma de Cono Adaptado de Martin (2012)	38
3.8. Modelo de Entidad Relacional MER	40
4.1. Logo de React	44
4.2. Logo de Django	45
4.3. Logo de PostgreSQL	45
4.4. Servicios Django, la carpeta logs es para almacenar información de cada servicio	47
4.5. Modelo Dermatologist, implementado en Django	48
4.6. Serializer utilizado como DTO	49
4.7. Serializer para crear una solicitud de dermatólogo	49
4.8. View para peticiones Post	50
4.9. View para listado de usuarios	51
4.10. Generación del token	52
4.11. Uso de user_id proveniente del JWT	54
4.12. Arquitectura basada en características en el frontend DermaUH	55

4.13. Configuración Axios para establecer comunicación	56
4.14. Servicios de la entidad Paciente	57
4.15. Hook para los servicios de pacientes	58
4.16. Matriz de confusión del modelo	60
5.1. Formulario de Registro	62
5.2. Página del Administrador	62
5.3. Usuarios Pendientes	63
5.4. Formulario Activar Usuario	63
5.5. Mensaje de que el usuario fue activado con éxito	64

Capítulo 1

Introducción

Desde tiempos inmemorables se ha buscado una forma de vincular las ciencias médicas con el desarrollo tecnológico, mejorando o creando herramientas digitales que permitan mejores resultados y ayuden incluso a doctores con gran experiencia. La dermatología, rama encargada de estudiar enfermedades producidas sobre la superficie de la piel, es una interesante propuesta para llevar a cabo esta conexión técnico-medicinal. El dermatólogo debe dar seguimiento a las lesiones cutáneas de sus pacientes, de esa forma si estos poseen una enfermedad grave, será diagnosticada a tiempo, elevando las posibilidades de cura o evolución positiva.

Para este diagnóstico y seguimiento, una herramienta fundamental es el dermatoscopio, un instrumento óptico que permite visualizar estructuras de la piel no visibles a simple vista, ampliando las imágenes y eliminando el reflejo de la luz superficial. Las imágenes dermatoscópicas resultantes son el insumo principal para el análisis de lesiones pigmentadas y no pigmentadas, ya que revelan patrones y características morfológicas clave (como atipias, redes pigmentarias o vasos anormales) que orientan al especialista hacia un diagnóstico más certero. De esta forma, la combinación del dermatoscopio con el registro digital de imágenes ha abierto la puerta a soluciones automatizadas que asisten al médico en la detección temprana de patologías como el melanoma.

Detrás de estas soluciones automatizadas se encuentra la inteligencia artificial, una disciplina que busca replicar capacidades humanas mediante sistemas computacionales. Para comprender el fundamento de estas soluciones y su aplicación en el análisis de imágenes dermatológicas, es necesario partir de una definición formal.

Según los autores del libro de referencia en la materia, Stuart Russell y Peter Norvig, la Inteligencia Artificial[1] se define como el estudio de agentes computacionales que operan de forma autónoma, perciben su entorno, se adaptan al cambio y persiguen objetivos. Como un subcampo de esta, el Machine Learning (o aprendizaje automático) es el estudio de algoritmos que permiten a los programas informáticos

mejorar automáticamente su rendimiento a través de la experiencia[2]. En esencia, el aprendizaje automático dota a los sistemas de la capacidad de aprender sin ser programados explícitamente para cada tarea.

Con el desarrollo y crecimiento de la Inteligencia Artificial (IA), específicamente el Aprendizaje de Máquinas (ML, por sus siglas en inglés) se han desarrollado diversas herramientas que ayudan no solo a dermatólogos, sino al personal de salud en general, en sus diagnósticos a pacientes. Hoy en día el procesamiento de imágenes ha abierto nuevas puertas que permiten a los médicos dar un seguimiento más sencillo y cómodo de las distintas lesiones, disminuyendo la fatiga visual y el uso de equipos externos, además de ser usado para adiestramiento de futuros doctores. Es importante destacar que todas estas herramientas constituyen un apoyo, no sustituyen a los especialistas.

1.1. Motivación

La medicina tiene vital importancia para la sociedad, por ese motivo, lograr que los médicos tengan una herramienta que les facilite su trabajo, además de ser un reto para cualquier persona apasionada por la tecnología, constituye una motivación tanto social como humana. En el contexto cubano, es de gran ayuda informatizar la mayor cantidad de procesos, debido a las dificultades logísticas que muchas veces se presentan. Dotar al personal de la salud de una aplicación web que les permita desde la comodidad de su teléfono dar seguimiento a sus pacientes y verificar si presentan alguna lesión grave con tan solo una fotografía, provocaría un gran avance en el sector medicinal de la nación. Esa herramienta ya existe desde hace varios años, concretamente desde 2023 con el desarrollo del sitio web DermaUH, el cual era capaz de suplir todas estas necesidades mencionadas y brindar las comodidades planteadas. Sin embargo, con el alto crecimiento que han tenido las herramientas de IA y los modelos de ML, acompañado de la falta de un buen conjunto de imágenes de pieles cubanas para entrenar el modelo desarrollado en su momento, se hace necesario desarrollar una nueva versión de DermaUH y vincularle nuevos modelos más potentes con altos porcentajes de efectividad clasificando imágenes de pieles cubanas.

1.2. Antecedentes

El ecosistema DermaUH ha evolucionado a través de dos hitos fundamentales previos a la presente versión 3.0.

Versión 2.0: Sitio web DermaUH (Mauricio Salim Mahmud Sánchez, 2024)

La versión 2.0 consistió en una plataforma web de apoyo al diagnóstico dermatoscópico, desarrollada como una aplicación de una sola página (SPA) con arquitectura

cliente-servidor. El frontend se construyó con Vue.js y TypeScript, mientras que el backend implementó una API RESTful construida con Django y Django Rest Framework, utilizando PostgreSQL como sistema gestor de base de datos. La plataforma permite a los médicos registrarse (con verificación por administradores), gestionar pacientes (CRUD completo), registrar lesiones cutáneas asociadas a cada paciente, y almacenar imágenes clínicas y dermatoscópicas por cada lesión. Entre sus funcionalidades avanzadas se incluyen: clasificación automática de lesiones mediante un ensemble de redes neuronales convolucionales (previamente desarrollado por Claudia Olavarrieta), segmentación de imágenes mediante arquitectura U-Net (desarrollada por Marcos Valdivié), y un sistema de colaboración que permite compartir lesiones entre especialistas para recibir diagnósticos adicionales. La plataforma también expone una API para fines investigativos, accesible por usuarios con permisos especiales. Todas las herramientas utilizadas son de código abierto. Esta versión sentó las bases del ecosistema, pero dependía completamente de conectividad a internet y estaba diseñada exclusivamente para uso en navegador web.

Versión móvil: Aplicación offline con sincronización eventual (Lázaro David Alba Ajete, 2025) Posteriormente, se desarrolló una aplicación móvil nativa (Android) que extiende el ecosistema al entorno offline. Implementada con Flutter y Dart, sigue los principios de Clean Architecture y el patrón MVVM. Utiliza Isar DB como base de datos local embebida y Riverpod para la gestión de estado. La aplicación permite a los dermatólogos, en ausencia de conexión a internet, registrar pacientes, capturar imágenes de lesiones (clínicas y dermatoscópicas), y ejecutar un modelo de clasificación de lesiones (Melanoma, Carcinoma Basocelular, Carcinoma Escamoso y Otros¹) mediante TensorFlow Lite embebido en el dispositivo. Los datos se almacenan localmente con campos de sincronización (`synced`, `deleted`, `remoteId`) y se transmiten de forma unidireccional al servidor central de DermaUH cuando se restablece la conectividad, mediante la API REST preexistente. La sincronización puede ser manual o automática diaria configurable. Esta versión resolvió el problema de la dependencia de internet, pero su modelo de IA es únicamente clasificador (sin capacidades explicativas) y el procesamiento de imágenes se limita a la inferencia del modelo previamente entrenado.

1.3. Formulación del problema

DermaUH pretende ser una herramienta potente de apoyo a especialistas en dermatología para el diagnóstico de las lesiones más frecuentes de cáncer de la piel. La

¹Se refiere a cualquier lesión no incluida en las tres mencionadas

versión 2.0, que actualmente corre, ha sido evaluada y se han detectado varios problemas que implicarán desarrollar nuevamente el frontend y el backend de la misma. Además el modelo actual de diagnóstico, que es un ensemble de 3 redes neuronales no fue entrenado con imágenes cubanas. Por este motivo se desea desarrollar una nueva versión del sitio DermaUH, la cual tendrá como principal mejora, la incorporación de nuevos modelos de procesamiento de imágenes. A continuación se expondrán las limitantes y/o deficiencias detectadas en las versiones anteriores del sitio web DermaUH.

1. El modelo de ML utilizado para clasificar imágenes no se entrenó con imágenes de pieles cubanas, alejándose en ese aspecto del cliente objetivo, los dermatólogos cubanos.
2. El diagnóstico asistido no viene acompañado de una justificación que responda a la interrogante ¿Por qué se obtuvo este resultado?.
3. El frontend de la aplicación tarda alrededor de diez segundos en cargar todos los elementos, haciendo lenta y molesta la experiencia del usuario.

Por lo tanto, el problema que aborda este trabajo se formula de la siguiente manera: la versión 2.0 de DermaUH presenta limitaciones en el modelo de clasificación (entrenado sin imágenes cubanas), la ausencia de explicabilidad en los diagnósticos, y deficiencias de rendimiento en el frontend. A partir de esto, se plantea la siguiente pregunta científica:

¿Es posible desarrollar una versión 3.0 de DermaUH que incorpore un nuevo modelo de procesamiento de imágenes (entrenado con pieles cubanas), un modelo explicativo de los diagnósticos, y mejore el rendimiento y la seguridad de la plataforma?

1.4. Objetivos Generales y Específicos

Como objetivo general se tendrá desarrollar la versión 3.0 de DermaUH, la cual tendrá un nuevo modelo de procesamiento de imágenes entrenado con imágenes de pieles cubanas, mejorará el rendimiento de la aplicación tanto del frontend como el backend, mejorará el protocolo de seguridad y manejo de solicitudes y notificaciones mediante un cliente de correo Email. Además esta versión será validada en 2027 en ensayo clínico. De aquí se derivan varios objetivos específicos:

1. Analizar herramientas existentes para el trabajo con imágenes dermatoscópicas así como sus ventajas y desventajas para el contexto cubano.

2. Lograr el desarrollo del frontend y backend que supere las deficiencias presentadas en la v2.0.
3. Mejorar las métricas de los modelos desarrollados e incorporarlos a la app una vez que sean reentrenados con el nuevo conjunto de imágenes de pieles cubanas.
4. Lograr una app web que clasifique y explique al especialista la decisión del diagnóstico mediante el modelo explicativo (XAI) desarrollado por la Lic.Amanda Noris[3].

1.5. Justificación

Se cuenta con una alta capacidad para el desarrollo de la versión 3.0 de DermaUH, debido a todos los conocimientos que se tienen sobre el stack tecnológico seleccionado y los adquiridos durante la carrera de Ciencias de la Computación. Además de eso existe abundante bibliografía sobre los temas abordados (IA, ML, Procesamiento de imágenes), lo cual facilita mucho la búsqueda de información para el desarrollo. Por último, la facultad MatCom ya tiene experiencia en el desarrollo de aplicaciones con un alto componente de IA, sobre todo en aquellas que su principal cualidad está vinculada al procesamiento de imágenes.

1.6. Estructura del Trabajo

El presente trabajo se organiza en cuatro capítulos, además de las conclusiones y recomendaciones finales.

- **Capítulo I. Estado del Arte:** Se analizan sistemas y plataformas existentes para el diagnóstico dermatológico asistido por IA, destacando sus ventajas y limitaciones en el contexto cubano. También se revisan técnicas de procesamiento de imágenes y modelos explicativos (XAI).
- **Capítulo II. Análisis y Diseño:** Se presentan los requisitos funcionales y no funcionales, los diagramas de casos de uso, la arquitectura del sistema (capas, cliente-servidor, API REST) y el diseño detallado de la base de datos.
- **Capítulo III. Implementación:** Se describen las tecnologías utilizadas, la integración de los modelos de IA y procesamiento de imágenes, y las decisiones de implementación más relevantes.
- **Capítulo IV. Pruebas y Resultados:** Se muestran las pruebas funcionales realizadas, las métricas de rendimiento y la validación del sistema con especialistas.

Finalmente, se exponen las conclusiones del trabajo y las recomendaciones para futuras líneas de desarrollo.

Capítulo 2

Estado del Arte

Para fundamentar la solución propuesta en esta tesis, resulta imprescindible analizar el panorama actual de las herramientas digitales y los modelos computacionales aplicados al diagnóstico dermatológico. El presente capítulo aborda, en primer lugar, una revisión de plataformas comerciales ampliamente utilizadas a nivel internacional, como DermEngine, SkinVision y VisualDx, evaluando sus características funcionales, costos de acceso y limitaciones para su implementación en el contexto cubano. En segundo lugar, se examinan sistemas académicos de referencia, incluyendo el archivo ISIC (International Skin Imaging Collaboration) y el conjunto de datos HAM10000, así como los modelos de aprendizaje automático derivados de ellos. A continuación, se describen las técnicas más relevantes de aprendizaje de máquinas en dermatología, el aprendizaje por transferencia y los enfoques de inteligencia artificial explicable (XAI), estos últimos particularmente importantes dotar al sistema de un modelo que justifique sus diagnósticos. Finalmente, se identifica una brecha clara entre las soluciones existentes y los requerimientos específicos que debe cumplir DermaUH, lo que justifica el desarrollo de una nueva versión adaptada a las condiciones de conectividad, disponibilidad de datos y necesidades de los dermatólogos cubanos.

2.1. Herramientas existentes

En esta sección se realizará un análisis de algunas de las aplicaciones que se utilizan para brindar ayuda en los diagnósticos dermatológicos. Dicho análisis se efectuará atendiendo a sus funcionalidades principales, ventajas y desventajas para el contexto cubano de cada aplicación móvil o web.

Criterios de selección

Para la selección de las herramientas analizadas en profundidad, se establecieron los siguientes criterios de inclusión: (1) disponibilidad comercial o acceso público desde al menos 2020, año a partir del cual se generalizó el uso de redes neuronales convolucionales en dermatología; (2) uso explícito de inteligencia artificial (preferentemente redes neuronales convolucionales) para la clasificación o análisis de lesiones cutáneas; (3) presencia de estudios clínicos publicados o certificaciones regulatorias (como marcado CE o aprobación de la FDA); (4) representatividad de diferentes modelos de negocio (suscripción, pago por uso, etc.). Con base en estos criterios, se seleccionaron tres herramientas: **DermEngine**, **SkinVision** y **VisualDx**.

Existen otras aplicaciones como Miiskin o First Derm, pero quedaron excluidas por no cumplir con el criterio (3) al carecer de estudios clínicos publicados que respalden su eficacia diagnóstica en el contexto dermatológico, o por estar orientadas exclusivamente a la teleconsulta sin componente de IA propio.

Las herramientas escogidas tienen en común el uso de modelos de aprendizaje automático como principal engranaje para asistir en los diagnósticos. Además, cada una cuenta con su propia base de imágenes, lo que permite un análisis comparativo de este aspecto tan importante, el cual forma parte del objetivo principal de esta tesis.

2.1.1. DermEngine



Figura 2.1: Logo de DermEngine

DermEngine[4] es una plataforma de software inteligente diseñada específicamente para dermatología, cuyo principal objetivo es optimizar el flujo de trabajo clínico mediante la integración de inteligencia artificial en el análisis de imágenes dermatoscópicas. Entre sus funcionalidades más relevantes se encuentran (estudio realizado en Marzo del 2026):

1. **Búsqueda visual inteligente (Visual Search):** Utiliza un algoritmo de inteligencia artificial que compara una imagen de una lesión con una base de datos de miles de casos etiquetados. A partir de esta comparación, el sistema genera una lista de posibles diagnósticos y ofrece una estimación del riesgo de malignidad, actuando como una segunda opinión automatizada para el especialista.

2. **Gestión integral de pacientes y lesiones:** Permite registrar y organizar la información clínica de los pacientes, incluyendo imágenes de sus lesiones, historiales de seguimiento y notas clínicas.
3. **Integración con dispositivos de captura:** Se conecta con MoleScope, un dermatoscopio digital de bajo costo que se acopla a teléfonos inteligentes.
4. **Imagenología de cuerpo completo:** Ofrece herramientas para mapear la superficie corporal del paciente y documentar la ubicación exacta de cada lesión.
5. **Colaboración y teledermatología:** Permite compartir casos entre especialistas de manera segura, obtener segundas opiniones y realizar sesiones de teleconsulta.

Ventajas

1. **Optimización del diagnóstico:** La IA proporciona una referencia rápida que puede aumentar la confianza del especialista.
2. **Herramienta educativa:** La base de datos de imágenes etiquetadas puede utilizarse para la formación de residentes y estudiantes.
3. **Seguimiento longitudinal:** La comparación de imágenes a lo largo del tiempo es crucial para la detección precoz de melanomas.
4. **Posible integración con dispositivos móviles:** Compatibilidad con MoleScope para captura estandarizada.

Desventajas

1. **Dependencia de conectividad a internet:** Plataforma basada en la nube, requiere conexión estable y rápida.
2. **Costo de acceso:** Suscripción desde 99 USD mensuales, inaccesible para dermatólogos cubanos.
3. **Barreras idiomáticas:** Interfaz y documentación principalmente en inglés.
4. **Falta de adaptación a la población cubana:** Modelos entrenados mayoritariamente con pieles claras, sin validación en fototipos II-VI (pieles más oscuras).



Figura 2.2: Logo de SkinVision

2.1.2. SkinVision

SkinVision es una aplicación móvil de servicio médico de pago, clínicamente validada, cuyo principal objetivo es la detección temprana del cáncer de piel enfocada en el autocuidado. Utiliza una red neuronal convolucional ResNet-50 para analizar imágenes de lunares o manchas cutáneas, clasificando las lesiones en riesgo alto, bajo o en revisión.



Figura 2.3: Estudio clínico de SkinVision

El proceso de evaluación consta de cuatro etapas: captura guiada de la imagen, análisis por IA (sensibilidad 92.1 % para melanoma, 98.6 % para carcinoma escamoso), revisión secundaria por expertos en casos de riesgo alto o en revisión, y entrega de resultados al usuario.



Figura 2.4: Diagrama del algoritmo de SkinVision

Ventajas

1. **Alta sensibilidad diagnóstica:** Reporta hasta 95% de sensibilidad para lesiones malignas.
2. **Validación clínica y regulatoria:** Certificación CE como dispositivo médico Clase IIa.
3. **Alta especificidad:** 89% en métricas recientes, reduciendo falsas alarmas.
4. **Revisión por expertos:** Control de calidad adicional para casos de alto riesgo.

Desventajas

1. **Enfoque en el paciente, no en el especialista:** No provee herramientas para gestión clínica.
2. **Costo de acceso:** Plan anual de aproximadamente €49.99, inaccesible para el sistema de salud cubano.
3. **Dependencia de conectividad:** Requiere internet para procesar imágenes en sus servidores.

4. **Falta de adaptación a pieles cubanas:** Entrenado mayoritariamente con pieles claras europeas, riesgo de sesgo.
5. **Potencial de falsa seguridad:** Un resultado de bajo riesgo podría retrasar una consulta médica necesaria.

2.1.3. VisualDx



Figura 2.5: Logo de VisualDx

VisualDx[5] es un sistema de apoyo a la decisión clínica que combina inteligencia artificial con una extensa biblioteca de imágenes médicas. Asiste a médicos no especialistas (atención primaria, urgencias) en el diagnóstico de enfermedades de la piel, reduciendo derivaciones a dermatología entre 25 % y 45 %.

Ventajas

1. **Enfoque en atención primaria:** Útil en contextos con acceso limitado al especialista. Los usuarios podrán realizar consultas “menores” en caso de no poder asistir a la consulta del doctor.
2. **Biblioteca inclusiva de pieles oscuras:** Ha trabajado durante 20 años en incluir fototipos II-VI.
3. **Apoyo al diagnóstico diferencial:** Basado en síntomas y demografía del paciente.
4. **Integración de IA y evidencia médica:** El usuario puede tomar una fotografía con su celular, no neceseramente con el dermatoscopio. Acto seguido rellena información sobre el paciente, como profesión, exposición al sol y antecedentes. La IA de VisualDx combina la imagen, con esa información y genera un diagnóstico que luego será validado por el especialista.

Desventajas

1. **Costo prohibitivo:** Suscripciones entre 299.99 y 649.99 euros anuales.
2. **Dependencia de internet:** Requiere conexión constante para acceder a la biblioteca y la IA.

3. **No diseñada para especialistas:** Orientada a médicos generales, no a dermatólogos para su práctica diaria.

2.1.4. Conclusiones del análisis de herramientas

En esta sección se analizaron herramientas con fines similares a DermaUH 3.0. A partir del análisis anterior, se presenta la Tabla 2.1 con un resumen comparativo.

Tabla 2.1: Comparativa de herramientas comerciales de diagnóstico dermatológico asistido por IA

Herramienta	Tipo de IA	Conjunto de datos (origen / fototipos)	Métricas reportadas	Costo (anual aprox.)	Dependencia internet	Lección para DermaUH
DermEngine	CNN (Visual Search)	Mayormente pieles claras (Europa/EE.UU.)	No especifica públicamente	\$1188 USD	Alta (nube)	Evitar dependencia total de la nube
SkinVision	ResNet-50	Mayormente pieles claras (Europa)	Sensibilidad 95 %, Especificidad 89 %	€50	Alta	La alta sensibilidad no es suficiente; se necesita explicabilidad
VisualDx	DermExpert (propietario)	Incluye pieles oscuras (28.5 %) pero con sesgo	Reducción derivaciones 25-45 %	\$299-649	Alta	Inclusión de pieles oscuras es insuficiente sin validación local

A partir del análisis se extraen los siguientes hechos concretos:

1. **Conjunto de imágenes:** VisualDx incluye pieles oscuras, pero las métricas de sus modelos son muy altas cuando debe clasificarlas; DermEngine y SkinVision usan mayoritariamente pieles claras, las pieles cubanas son en su mayoría mestizas u oscuras, por tanto el hecho de tener solamente etnias claras provocaría un sesgo y una bajada en las métricas al analizar pieles cubanas.
2. **Costo de suscripción:** Períodos de prueba gratuitos breves (7-30 días). Las suscripciones son inaccesibles para el sistema de salud cubano.

3. **Modelo explicativo:** Ninguna herramienta incorpora inteligencia artificial explicable (XAI) que justifique el diagnóstico.
4. **Rendimiento:** DermEngine incluye un video de 3 minutos en su portada; Skin-Vision abusa de animaciones y videos, degradando el rendimiento sin conexión rápida.

Lecciones para DermaUH 3.0

Del análisis anterior, DermaUH 3.0 debe adoptar las siguientes decisiones de diseño:

- **Modelo entrenado con pieles cubanas:** Buscar más representatividad de los fototipos del IV-VI para el conjunto de datos de entrenamiento, los cuales son los predominantes en la población cubana.
- **Explicabilidad diagnóstica:** Incorporar un modelo XAI que justifique cada predicción, aumentando la confianza del especialista.
- **Gratuidad total:** La plataforma será completamente gratuita para el sistema de salud cubanos, eliminando la barrera económica.
- **Optimización de rendimiento:** Priorizar un diseño ligero, con tiempos de carga optimizados y consumo eficiente de recursos, evitando videos o animaciones pesadas.

Estas lecciones reflejan la importancia de desarrollar una aplicación para el diagnóstico asistido por IA, libre de costo, con modelos entrenados con imágenes de pieles cubanas, que integre un modelo explicativo y que funcione adecuadamente en las condiciones de conectividad del sistema de salud cubano.

2.2. Archivos académicos de referencia

Además de las herramientas comerciales, es necesario estudiar los repositorios académicos que sirven como base para el entrenamiento de modelos de IA:

2.2.1. ISIC Archive: Repositorio académico de referencia

La **International Skin Imaging Collaboration (ISIC) Archive**[6] es un repositorio público de acceso abierto que contiene una de las colecciones más grandes y completas del mundo de imágenes dermatoscópicas y metadatos clínicos asociados. Concebido para apoyar la docencia, la investigación y el desarrollo de algoritmos de

inteligencia artificial, el archivo se ha convertido en el banco de pruebas estándar para sistemas de diagnóstico asistido en dermatología. Las imágenes provienen de múltiples centros internacionales y han sido etiquetadas mediante confirmación histopatológica (el estándar de oro) o por consenso de múltiples expertos, lo que garantiza un alto grado de fiabilidad diagnóstica. Su impacto trasciende lo puramente científico, pues las competiciones anuales conocidas como *ISIC Challenges* han permitido a la comunidad de ciencia de la computación contribuir activamente a la mejora de estos sistemas, moviendo los límites del rendimiento de los modelos de aprendizaje automático.

Organización de los datos

Los datos de la ISIC Archive están organizados siguiendo una jerarquía de categorías y marcos taxonómicos que facilitan el desarrollo de modelos de inteligencia artificial. Las Tablas 2.2 y 2.3 detallan las principales vías de clasificación dentro del ecosistema: un marco general de clasificación por diagnóstico (utilizado tanto por los médicos como por el dataset HAM10000 integrante del archivo) y un marco orientado al desarrollo de algoritmos de *machine learning*, que incorpora la atribución de diagnóstico para problemas multiclase.

Tabla 2.2: Principales categorías generales de diagnóstico de lesiones en el archivo ISIC y el dataset HAM10000

Categoría General	Descripción	Ejemplos de Subcategorías
Lesiones Melanocíticas	Lesiones pigmentadas originadas de melanocitos.	Nevo Melanocítico (Benigno), Melanoma (Maligno)
Lesiones Queratinocíticas	Lesiones derivadas de células queratinizantes.	Queratosis Actínica / Enfermedad de Bowen, Queratosis Benignas (queratosis seborreica)
Neoplasias Vasculares / Raras	Lesiones con origen en vasos sanguíneos o de presentación poco común.	Lesiones Vasculares, Dermatofibroma
Carcinoma de Células Basales (BCC)	Tumor maligno de células basales, el más común.	Carcinoma Basocelular (BCC)

Nota: El número de clases presentadas y su agrupación puede variar ligeramente entre las diferentes versiones del dataset (por ejemplo, 2018, 2019).

Tabla 2.3: Metadatos asociados a los diagnósticos y marco de tareas para la comunidad de Machine Learning

Caso de Uso / Metadato	Nivel Jerárquico	Atributos y Descripción
Atributos de Diagnóstico de Alto Nivel	Diagnóstico de Lesión	Benigna: Nevo Melanocítico, Queratosis Benigna, Dermatofibroma. Indeterminada/Pre-Maligna: Queratosis Actínica / Enfermedad de Bowen. Maligna: Melanoma, Carcinoma Basocelular (BCC).
Diagnóstico Específico	Lesión Específica	Permite clasificaciones de hasta quinto nivel, identificando patologías concretas según la taxonomía ISIC-IDDX.
Confirmación del Diagnóstico (diagnosis confirm type)	Mecanismo de Verificación	Identifica si la etiqueta se obtuvo por confirmación histopatológica o por consenso de expertos .
Subtareas de la Competencia ISIC	Problema de ML	Segmentación: Delimitación de los bordes de la lesión. Detección de Atributos Dermoscópicos (ATZ). Clasificación: Distinción entre lesiones benignas y malignas.

Nota: `diagnosis_confirm_type` “histopathology” se refiere a la confirmación mediante biopsia o análisis de tejido; “consensus” implica un acuerdo diagnóstico entre múltiples dermatólogos expertos sin necesidad de biopsia.

Este andamiaje de datos, etiquetas y desafíos competitivos ha posicionado a la ISIC Archive como el estándar de facto para la comparación de algoritmos, impulsando el progreso en la detección automatizada del melanoma a nivel mundial. Además de eso existen numerosos conjuntos de datos (Datasets), derivados de ISIC, que han sido utilizados en artículos científicos, proyectos de ML e investigaciones relacionadas con la IA. A continuación una mención y breve explicación de algunos de estos datasets.

2.2.2. DERM12345

Según un artículo presentado por la revista Nature (una de las más prestigiosas revistas científicas a nivel mundial, fundada por el astrónomo británico Joseph Norman Lockyer.), presenta un conjunto de datos diversos que comprende 12.345 imágenes dermatoscópicas con 40 subclases de lesiones cutáneas, recogidas en Turkiye, que comprende diferentes tipos de piel en la zona de transición entre Europa y Asia. Cada subgrupo contiene imágenes de alta resolución y anotaciones de expertos, proporcionando una base sólida y confiable para futuras investigaciones. El análisis detallado de cada subgrupo proporcionado en este estudio facilita los esfuerzos de investigación dirigidos y mejora la profundidad de la comprensión con respecto a las lesiones de la piel. Este conjunto de datos se distingue a través de una estructura diversa con sus 5 súper clases, 15 clases principales, 40 subclases y 12.345 imágenes dermatoscópicas de alta resolución[7].

2.2.3. HAM10000

Otro artículo presentado en la revista Nature, referido al dataset HAM10000 expresa que el conjunto de datos final consta de 10015 imágenes dermatoscópicas que se publican como un conjunto de capacitación para fines académicos de aprendizaje automático y están disponibles públicamente a través del archivo de la ISIC. Este conjunto de datos de referencia se puede utilizar para el aprendizaje automático y para comparaciones con expertos humanos. Los casos incluyen una colección representativa de todas las categorías de diagnóstico importantes en el ámbito de las lesiones pigmentadas. Más del 50% de las lesiones han sido confirmadas por patología, mientras que la verdad fundamental para el resto de los casos fue el seguimiento, el consenso de expertos o la confirmación por microscopía confocal en vivo[8].

2.2.4. Conclusiones parciales sobre los archivos académicos de referencia

Aunque estos dataset son altamente reconocidos y utilizados en disímiles de aplicaciones a nivel mundial, no cumplen con los requisitos del conjunto de datos necesarios en la versión 3.0 de DermaUH. Por ejemplo el conjunto de datos DERM12345, recopiló información mayormente en Turkiye, específicamente la zona de transición entre Europa y Asia en las cuales existen factores externos que pueden provocar en las lesiones cutáneas registradas un comportamiento y evolución diferente al esperado en pieles cubanas, dichos factores pueden ser intensidad del sol en esta parte del planeta, el clima y las temperaturas[9]. Por otro lado el dataset HAM10000, está conformado por un alto número de pieles claras, provocando que los modelos ML que se pueden entrenar tengan un alto sesgo de este tipo de piel.

2.3. Técnicas de inteligencia artificial explicable XAI en dermatología

Según la tesis de la Licenciada Amanda Noris[3], la inteligencia artificial explicativa (XAI) surge como respuesta al creciente uso de modelos de IA cuyos procesos de toma de decisiones son difíciles de interpretar. Los métodos XAI se dividen en dos grandes enfoques:

- Modelos intrínsecamente explicables: Buscan diseñar modelos cuya estructura sea comprensible por sí misma.
- Técnicas de explicación post-hoc: Interpretan modelos ya entrenados.

Capítulo 3

Metodología y Análisis del Sistema Propuesto

El análisis del estado del arte presentado en el capítulo anterior evidenció las limitaciones de las herramientas comerciales y académicas existentes para el diagnóstico dermatológico asistido por inteligencia artificial, particularmente en lo referente a la falta de adaptación a los fototipos de piel oscura, los altos costos de suscripción, la dependencia de conectividad a internet y la ausencia de modelos explicativos (XAI). A partir de estas brechas identificadas, el presente capítulo describe la solución propuesta para DermaUH versión 3.0. En primer lugar, se define la metodología de desarrollo de software que guiará el proyecto, justificando la elección de un enfoque ágil adaptado a los requisitos del sistema. A continuación, se presenta la especificación de requisitos funcionales y no funcionales, estableciendo las capacidades que debe ofrecer la plataforma y las condiciones de calidad que debe cumplir. Finalmente, se aborda el análisis y diseño del sistema, incluyendo el modelado del negocio mediante diagramas de casos de uso UML, el diseño arquitectónico (capas, cliente-servidor, MVC) acompañado de su diagrama de componentes, y el diseño detallado con diagramas de secuencia y el modelo de base de datos (diagrama entidad-relación y modelo relacional normalizado). De esta forma, se sientan las bases técnicas para la implementación de una solución que responda a las necesidades específicas de los dermatólogos cubanos.

3.1. Metodología

Una metodología de desarrollo de software es un marco de trabajo que se usa para estructurar, planificar y controlar el proceso de desarrollo de sistemas de información. Las metodologías de Desarrollo de Software (DS.) han experimentado un proceso histórico y evolutivo que inicia en los años 40 con la aparición de las primeras computadoras. Hoy en día existen dos tipos de metodologías, tradicionales y ágiles.

[10]

- 1- **Metodologías Tradicionales:** Centran su atención en llevar una documentación exhaustiva de todo el proyecto, planificación y control del mismo. Imponen una disciplina rigurosa, con el fin de lograr un software más eficiente. Algunas de estas metodologías son:

- 1.1- **Cascada:** Consiste en fases secuenciales donde cada una debe completarse antes de pasar a la fase siguiente. Es muy fácil de entender y gestionar, es ideal cuando los requisitos son estables y bien conocidos, sin embargo es muy rígida antes cambios y el cliente solo ve el producto final.

- 1.2- **V-Model:** Es una variante del Cascada que enfatiza la verificación y validación (V&V), cada fase de desarrollo tiene una fase de prueba correspondiente. Presenta una alta calidad y trazabilidad, además las pruebas se planifican desde el inicio. Por otro lado sigue siendo rígido como Cascada y no es apto para requisitos cambiantes.

- 1.3- **Modelo Espiral:** Propuesto por Barry Boehm, combina Cascada con gestión de riesgos mediante iteraciones que aumentan en detalle. Este enfoque permite iteraciones controladas y es bueno para proyectos grandes con incertidumbre técnica, sin embargo es complejo de gestionar y puede ser costoso en cuanto a recursos.

- 2- **Metodologías Ágiles:** Generalmente son procesos *Incrementales* (entregas en ciclos cortos y rápidos), *Cooperativo* (clientes y desarrolladores trabajan constantemente con una comunicación muy fina y constante), Sencillo (el método es fácil de aprender y modificar para el equipo) y finalmente Adaptativo (capaz de permitir cambios de último momento). Estas metodologías ponen de relevancia que la capacidad de respuesta a un cambio es más importante que el seguimiento estricto de un plan [11]. Entre las más conocidas se pueden apreciar:

- 2.1- **Scrum:** Se basa en ciclos de trabajos cortos llamados *Sprints* que tienen una duración de entre 2 y 4 semanas. Es ideal para proyectos complejos que requieren flexibilidad y entregas rápidas. [12]

- 2.2- **Kanban:** Kanban se centra en visualizar el flujo de trabajo y limitar el trabajo en progreso (WIP, por sus siglas en inglés). No utiliza iteraciones de tiempo fijo como Scrum, sino que se basa en un flujo continuo de tareas. Es ideal para equipos que buscan mejorar la eficiencia y la gestión de tareas, pero puede ser menos estructurado que Scrum.

- 2.3- **Extreme Programming (XP):** XP se enfoca en las prácticas de ingeniería de software para producir código de alta calidad y adaptarse a requisitos

cambiantes. Es especialmente adecuado para proyectos con requisitos muy volátiles. Promueve prácticas como la programación en parejas (dos desarrolladores trabajan en el mismo código), el desarrollo guiado por pruebas (TDD), la integración continua y las refactorizaciones frecuentes para mejorar el diseño del código.

Para el desarrollo de DermaUH 3.0 se decidió utilizar Scrum, debido a que es una metodología ágil que se adapta bien a proyectos complejos y con requisitos cambiantes, como es el caso de DermaUH. Además, Scrum permite entregas rápidas y frecuentes, lo que facilita la retroalimentación constante del cliente y la adaptación a sus necesidades. Por otro lado, el desarrollador tiene experiencia previa con esta metodología. Aunque se mencionan requisitos cambiantes DermaUH está pensado para mantenerse “plana” (es decir sin cambios sobresalientes) a largo plazo, lo cual no hace necesario el uso de Kanban o XP que están enfocadas en aplicaciones con requisitos volátiles. Para ello el proceso de desarrollo fue dividido en seis sprints, con una duración promedio de entre dos y tres semanas cada uno. Al final de cada sprint el equipo (dermatólogos y desarrollador) realizó una evaluación y validación de los resultados obtenidos. Los sprints en los que se dividió el proceso son los siguientes:

1. Desarrollo de toda la sección de Registro e Inicio de Sesión de los usuarios. Definir roles y permisos de cada uno.
2. Implementar un servicio de correo, el cual notifica al usuario administrador de nuevas solicitudes y a los usuarios les notifica la respuesta de su solicitud.
3. Desarrollar las funcionalidades básicas de la Aplicación, gestión de pacientes, lesiones y diagnósticos.
4. Entrenar los nuevos modelos a integrar con el nuevo conjunto imágenes entregado por las organizaciones de la salud.
5. Desarrollar el servicio de consulta de Imagen y vincular el modelo a la aplicación DermaUH 3.0
6. Realizar pruebas de rendimiento y ejecución con doctores para posteriormente lanzar la aplicación.

3.2. Levantamiento de Requerimientos

El levantamiento de requerimientos es el proceso mediante el cual el analista identifica, documenta y acuerda las necesidades, expectativas y restricciones que debe cumplir un sistema de software. Incluye tanto los requerimientos funcionales (qué debe

hacer el sistema) como los no funcionales (cómo debe hacerlo: rendimiento, seguridad, usabilidad, etc.). Este proceso involucra a los stakeholders (clientes, usuarios finales, desarrolladores) y constituye la base sobre la que se construye todo el proyecto.

Los requerimientos son importantes porque actúan como el contrato implícito entre el equipo de desarrollo y los interesados. Unos requerimientos claros y bien gestionados permiten: definir el alcance realista del proyecto, evitar malentendidos y retrabajos costosos, establecer criterios objetivos de validación y aceptación y priorizar funcionalidades según el valor que aportan. La falta de una buena gestión de requerimientos es una de las causas más frecuentes de fracaso en proyectos de software, ya que conduce a entregas que no satisfacen las necesidades reales, sobrecostes y demoras.

3.2.1. Requerimientos Funcionales

Según un artículo de Michel Arias Chaves, para la revista InterSede [13] los requerimientos funcionales son los que definen las funciones que el sistema será capaz de realizar, describen las transformaciones que el sistema realiza sobre las entradas para producir salidas. Es importante que se describa el ¿Qué? y no el ¿Cómo? se deben hacer esas transformaciones. Estos requerimientos al tiempo que avanza el proyecto de software se convierten en los algoritmos, la lógica y gran parte del código del sistema. Siguiendo este concepto, los requerimientos funcionales de DermaUH 3.0 son:

- RF1 El usuario autenticado puede hacer **CRUD** sobre las entidades Paciente, Lesión y diagnóstico.
- RF2 Los usuarios podrán obtener información de lesiones registradas, solo de sus propios pacientes, podrán acceder a otras lesiones si estas se hacen públicas o se comparten con el mismo.
- RF3 El usuario puede enviar una imagen dermatoscópica¹ o clínica al sistema y recibir una sugerencia de clasificación automatizada con nivel de confianza, la cual debe ser validada por un especialista médico antes de cualquier decisión clínica. En ningún caso el diagnóstico automático reemplaza al especialista; su función es únicamente de apoyo.
- RF4 Solamente el administrador tiene permiso para eliminar cuentas y activar usuarios en el sistema y consultar detalles de cualquier cuenta.
- RF5 Cualquier persona puede enviar una solicitud de registro a la aplicación, la cual deberá aportar datos profesionales como el hospital y el carnet de identidad

¹La dermatoscopia es una técnica de imagen no invasiva que amplía las estructuras de la piel, permitiendo observar patrones no visibles a simple vista.

para después será aprobada por el Administrador, el cual tiene acceso a las base de datos del Ministerio de Salud Pública (MINSAP).

3.2.2. Requerimientos No Funcionales

Los requerimientos no funcionales tienen que ver con características que de una u otra forma puedan limitar el sistema, como por ejemplo, el rendimiento (en tiempo y espacio), interfaces de usuario, fiabilidad (robustez del sistema, disponibilidad de equipo), mantenimiento, seguridad, portabilidad, estándares, etc. Siguiendo esta línea se concluye que los requerimientos no funcionales de DermaUH son:

- RNF1 La autenticación debe ser mediante email y password generando un JSON Web Token (JWT) el cual definirá los permisos del usuario que hace inicio de sesión. El JWT a su vez debe tener un tiempo de expiración y ser renovable
- RNF2 La comunicación entre cliente y servidor debe estar cifrada mediante HTTPS.
- RNF3 El tiempo de operación de una petición y la generación de la respuesta debe ser inferior a dos segundos con internet estable, bajo una carga de hasta 50 usuarios concurrentes.
- RNF4 La interfaz web debe ser responsive (adaptable a resoluciones de 320px a 1920px).
- RNF5 Cualquier respuesta a una solicitud, sea positiva o de error debe mostrarse con mensajes claros a los usuarios.

3.3. Análisis y Diseño

Una vez definidos los requisitos funcionales y no funcionales que debe cumplir el sistema, el siguiente paso consiste en modelar su estructura y comportamiento desde una perspectiva técnica. Este apartado presenta los artefactos de diseño que guiarán la implementación de la versión 3.0 de DermaUH. Se abordarán temas como: casos de uso que aparecen en la aplicación, arquitectura de software seleccionada así como estructura, diseño y modelación de las entidades de la base de datos.

3.3.1. Diagrama de Casos de Uso

Los diagramas de casos de uso son utilizados durante la fase de análisis de un proyecto para identificar y dividir la funcionalidad del sistema. Normalmente contienen: casos de uso, actores y relaciones entre ellos: de asociación, de dependencia y/o de generalización[14]. En otras palabras es una forma de esquematizar y representar los requerimientos funcionales planteados en el epígrafe anterior.

- **Casos de Uso:** Los casos de uso describen el comportamiento del sistema cuando uno de los actores envía un estímulo concreto.
- **Actores:** Los actores interpretan papeles que pueden ser representados por los usuarios del sistema. Dichos usuarios pueden ser humanos, otros ordenadores o incluso otros sistemas de software

A continuación se mostrarán los diagramas de casos de uso, realizados con la herramienta PlantUML debido a que permite mediante su propio lenguaje *plantuml* crear imágenes de diagramas de casos de uso. Normalmente en un diagrama de casos de uso se visualizan todos los actores y los casos de uso, por simplicidad y para que sea más entendible para el lector, se separaron los diagramas por actores.

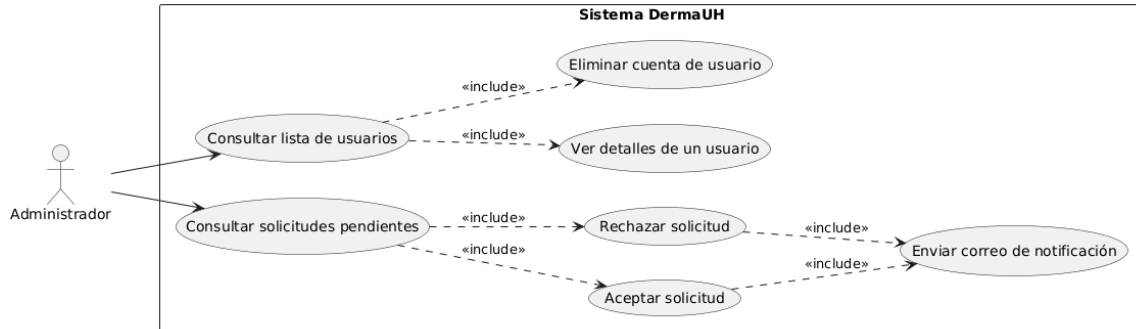


Figura 3.1: Diagrama UML del actor Admin

La figura 3.1 muestra las acciones permitidas cuando el usuario entra al sistema con permisos de administrador. A continuación una breve explicación de cada una:

En la pestaña *Usuarios*:

- **Listar Cuentas de Usuarios:** Para acceder a eliminar o ver detalles, el administrador primero debe consultar la lista de usuarios
- **Eliminar Cuenta del Usuario:** Esta opción permite eliminar la cuenta de un usuario de la aplicación, sin importar los permisos del usuario ni el tiempo que lleve activo en el sistema.
- **Ver Detalles del Usuario:** Esta opción le muestra al administrador todos los datos personales que el usuario ingresó en el formulario de registro, como su nombre completo, hospital al que pertenece, correo entre otros. En caso del usuario ingresar como residente, muestra también el correo del doctor encargado de tutorearlo.

En la pestaña *Solicitudes*:

- **Rechazar Solicitud de Usuario:** Con esta opción el administrador rechaza, lo cual es sinónimo de eliminar, al menos en este contexto, la solicitud de registro enviada por un usuario.
- **Aceptar Solicitud de Usuario:** Permite al administrador aceptar la solicitud de registro de un usuario y este ya podrá utilizar la aplicación en dependencia de los permisos asignados por el administrador.
- **Asignar Roles al Usuario:** Una vez aceptada la solicitud de registro, el administrador debe asignarle permisos a dicho usuario. Los permisos permitidos son administrador y/o doctor.

- **Enviar correo:** La acción de aceptar o rechazar una solicitud incluye automáticamente el envío de un correo al solicitante, por lo que se modeló como una relación de inclusión («include»)

El usuario con rol de administrador es el único facultado para gestionar las cuentas de usuario y las solicitudes de registro.



Figura 3.2: Diagrama UML del actor Doctor

- **Registrar Nuevos Pacientes:** En el panel principal de la aplicación, arriba a la derecha, aparece un botón para registrar pacientes nuevos en el sistema.

Al presionarlo se muestra un formulario que debe ser rellenado con información personal del paciente, nombre, edad, sexo, profesión etc. Una vez rellenado el formulario, se presiona el botón de finalizar registro y el paciente queda registrado en el sistema.

- **Ver detalles de un Paciente:** Los pacientes son representados como tarjetas en el panel principal. A la derecha de cada tarjeta se muestra un botón con la etiqueta *Detalles* que permite al doctor visualizar los datos personales del paciente, separada en tres secciones, información personal, información étnica y dirección.
- **Ver lesiones de un Paciente:** Similar a la opción anterior, a la derecha del paciente se muestra el botón *Lesiones*. Este le permite al doctor observar el registro de lesiones del paciente en cuestión.
- **Registrar nuevas lesiones:** Se puede llegar a esta opción de dos formas diferentes, cuando se registra un nuevo paciente y se presiona el botón finalizar registro, aparece un nuevo formulario para registrar la nueva lesión y la otra es al acceder al listado de lesiones de un paciente se muestra un botón para agregar una lesión nueva.
- **Eliminar un Paciente:** Similar a ver lesiones de un paciente y ver detalles se accede a esta opción mediante un botón ubicado a la derecha de la tarjeta del paciente.
- **Listar y buscar pacientes mediante filtros:** En la parte superior del panel principal aparece una barra de búsqueda, donde el doctor podrá introducir una consulta y la aplicación filtrará a los pacientes por la consulta del doctor. Además aparece una opción para ordenar los pacientes por algún campo.
- **Compartir lesiones con otros doctores:** Con esta opción, el doctor tiene la posibilidad de compartir una lesión con otros colegas del personal de la salud. Los doctores escogidos para compartir tendrán la posibilidad de ver solamente el nombre y apellido del paciente y las imágenes de la lesión.
- **Solicitar diagnóstico a través de la imagen:** Esta opción permite al doctor tomar una fotografía, ya sea con el móvil o con el dermatoscopio y solicitar un diagnóstico aproximado de confianza proporcionado por un modelo de ML especializado en procesamiento de imágenes.

3.3.2. Diagramas de Secuencia

La Figura 3.4 muestra el diagrama de secuencia para el actor doctor.

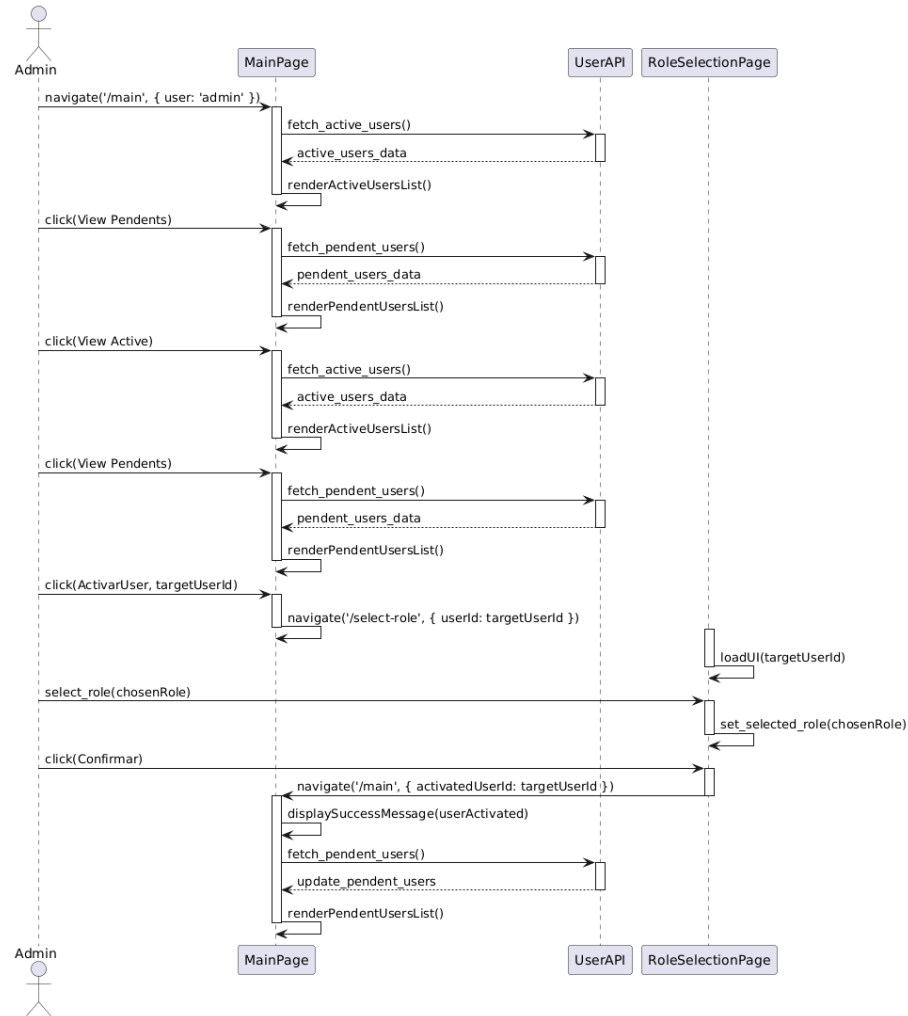


Figura 3.3: Diagrama de Secuencia del actor Admin

3.3.3. Análisis y propuesta arquitectónica

Una vez modelados los casos de uso que describen el comportamiento funcional del sistema desde la perspectiva de los actores, el siguiente paso consiste en definir la estructura organizativa que soportará dichas funcionalidades. La arquitectura de software se refiere a la disposición conceptual y estructural que subyace en el diseño y desarrollo de sistemas, estableciendo los componentes, sus relaciones y los principios que guían su evolución. Su importancia radica en proporcionar un marco de trabajo para cumplir con los requisitos funcionales y no funcionales (rendimiento, seguridad, escalabilidad), así como para tomar decisiones tecnológicas fundamentadas[15]. En este apartado se analizarán los patrones estructurales más conocidos atendiendo a

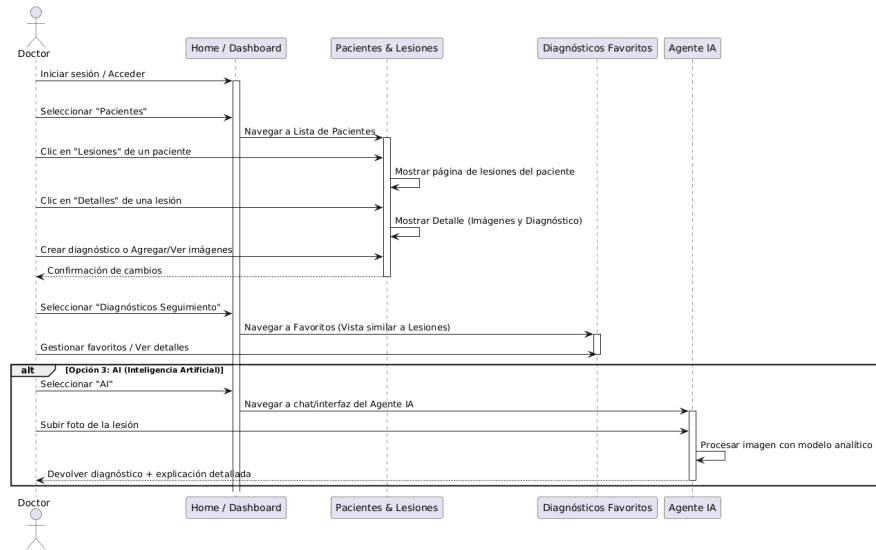


Figura 3.4: Diagrama de Secuencia del actor Doctor

su capacidad de escalabilidad, implementación para solucionar el problema en cuestión y los conocimientos adquiridos en la carrera de Licenciatura en Ciencias de la Computación.

Antes de analizar las alternativas, conviene distinguir tres niveles conceptuales:

- **Estilo arquitectónico:** define una familia de sistemas con una estructura común (ej. capas, microservicios).
- **Patrón de arquitectura:** resuelve un problema específico dentro de un estilo (ej. MVC, Hexagonal).
- **Arquitectura de software:** la organización concreta de un sistema particular, incluyendo decisiones tecnológicas.

En esta sección se analizan tanto estilos (capas) como patrones (hexagonal, cebolla, limpia) bien conocidos en la literatura.

Estilo arquitectónico por capas

Según un artículo publicado en el blog del sitio ReactiveProgramming, dedicado a la publicaciones relacionadas con el mundo de la programación y específicamente la Ingeniería de Software la arquitectura en capas consta en dividir la aplicación en niveles (capas), haciendo que cada uno tenga un rol muy definido[16]. Un ejemplo de esta división por niveles podría ser:

1. Presentación (Interfaz de Usuario)
2. Reglas de negocio (Servicios)
3. Acceso a datos (Base de Datos)

Una ventaja de este estilo arquitectónico es que no delimita el número de capas que el desarrollador puede utilizar y/o implementar, su idea fundamental la basa en la aplicación del Principio de Separación de Responsabilidades[17]. En la práctica, la mayoría de las veces este estilo arquitectónico es implementado en 4 módulos: presentación, negocio, persistencia y base de datos, sin embargo, es habitual ver que las capas de negocio y persistencia se combinan en una sola, sobre todo cuando la lógica de persistencia está incrustada dentro de la capa de negocio.

La estructura de una arquitectura por capas determina que cada nivel debe “colocarse” de forma horizontal uno sobre el otro, lo que restringe la comunicación directa únicamente al nivel inmediatamente inferior. En síntesis, si el nivel de presentación desea acceder al de acceso a datos, debe hacerlo únicamente a través de la capa de servicios. Esta última realiza la petición correspondiente a la capa de acceso a datos. Luego, la respuesta hace el recorrido inverso hasta llegar al módulo de presentación.

Como característica de este diseño: los niveles deben ser componentes apartes, es decir estar lo suficientemente desacoplados para que si se desea cambiar la lógica o el funcionamiento de alguno, no sea necesario aplicar cambios en el capas. En algunas ocasiones estas capas se alojan en servidores distintos pero que se comunican entre sí.

La separación de la aplicación en capas busca cumplir con el principio de separación de preocupaciones[17], de tal forma que cada nivel se encargue de una tarea muy definida, por ejemplo, el módulo de presentación solo se preocupa por presentar la información de forma agradable (en algunos casos se usa el término interfaz amigable) al usuario, pero no le interesa de donde viene la información ni la lógica de negocio que hay detrás, en su lugar, solo sabe que existe un estrato de negocio que le proporcionará la información. El nivel de abstracción de negocio solo se encarga de aplicar todas las reglas de negocio y validaciones, pero no le interesa como recuperar los datos, guardarlos o borrarlos, ya que para eso tiene una capa de persistencia que se encarga de eso. Por otro lado, el nivel de persistencia es el encargado de comunicarse con la base de datos, crear las instrucciones SQL para consultar, insertar, actualizar o borrar registros y retornarlos en un formato independiente a la base de datos. De esta forma, se concluye que cada capa tiene responsabilidad única y no necesita conocer el estado de los niveles superiores para poder llevar a cabo su función principal. A continuación veremos algunas ventajas y desventajas del uso de esta arquitectura.

Ventajas

1. **Separación de responsabilidades:** Permite la separación de preocupaciones (SoC), ya que cada capa tiene una sola responsabilidad.
2. **Fácil de desarrollar:** Este estilo arquitectónico es especialmente fácil de implementar, además de que es muy conocido y una gran mayoría de las aplicaciones la utilizan.
3. **Fácil de probar:** Permite probar cada nivel de forma aislada mediante el uso de dobles de prueba (mocks)[18] para las dependencias.
4. **Fácil de mantener:** Debido a que cada módulo hace una tarea muy específica, es fácil detectar el origen de un bug para corregirlo, o simplemente se puede identificar donde se debe aplicar un cambio.
5. **Seguridad:** La separación de capas permite el aislamiento de los servidores en subredes diferentes, lo que hace más difícil realizar ataques.

Desventajas

1. **Performance:** La comunicación por la red o internet es una de las tareas más tardadas de un sistema, incluso, en muchas ocasiones más tardado que el mismo procesamiento de los datos, por lo que el hecho de tener que comunicarnos de capa en capa genera un degrado significativo en el performance.
2. **Escalabilidad:** Las aplicaciones que implementan este patrón por lo general tienden a ser monolíticas, lo que dificulta su escalabilidad.
3. **Tolerancia a los fallos:** Si una capa falla, todas las capas superiores comienzan a fallar en cascada.

Patrón Hexagonal (Puertos y Adaptadores)

La arquitectura hexagonal, también conocida como arquitectura de puertos y adaptadores, fue propuesta por Alistair Cockburn en 2005 como una evolución de los patrones de arquitectura limpia[19]. Su principio fundamental es aislar el dominio (lógica de negocio) de los detalles técnicos externos como interfaces de usuario, bases de datos o servicios de mensajería. Para lograr este aislamiento, Cockburn introduce dos conceptos clave: los **puertos** y los **adaptadores**.

Puertos y Adaptadores Un **puerto** es una interfaz que define un conjunto de operaciones que el dominio ofrece o requiere. No contiene implementación alguna, solo la especificación de cómo comunicarse con el dominio. Por ejemplo, un puerto podría ser `RepositorioPaciente` (interfaz para guardar y recuperar pacientes), otro podría ser `ServicioNotificaciones` (interfaz para enviar correos electrónicos). Por otro lado, un **adaptador** es una implementación concreta de un puerto que conecta el dominio con una tecnología específica. Un adaptador concreto sería `RepositorioPacientePostgreSQL`, que implementa `RepositorioPaciente` usando SQL, o `RepositorioPacienteEnMemoria`, útil para pruebas.

La arquitectura recibe su nombre del hexágono imaginario que representa el dominio central. Cada lado del hexágono corresponde a un puerto, y los adaptadores se conectan a esos lados. No es necesario que haya exactamente seis puertos; el hexágono es una metáfora visual que sugiere que puede haber múltiples puntos de conexión sin que el dominio conozca los detalles de quién o qué se conecta a ellos[20].

Regla de Dependencia La regla cardinal de la arquitectura hexagonal es que las dependencias apuntan siempre hacia adentro. Es decir, el dominio central nunca depende de los adaptadores externos ni de los frameworks. En lugar de ello, los adaptadores dependen del dominio (a través de los puertos). Esto invierte la dependencia tradicional de las arquitecturas en capas, donde la capa superior depende de la inferior. El siguiente fragmento de código ilustra este principio:

```
// Puerto (interfaz) en el dominio
interface PatientRepository {
    Patient findById(PatientId id);
    void save(Patient patient);
}

// Adaptador (implementación) en la infraestructura
class PostgresPatientRepository implements PatientRepository {
    // Implementación con SQL
}
```

Gracias a esta separación, el dominio nunca depende de la base de datos ni del framework web. Esto facilita probar el dominio de forma aislada (con adaptadores de prueba como `InMemoryPatientRepository`), cambiar de base de datos sin tocar la lógica de negocio (solo se escribe un nuevo adaptador), y retrasar decisiones tecnológicas (se puede comenzar con un adaptador en memoria y luego reemplazarlo por uno real).

Ventajas específicas Además de la independencia tecnológica, la arquitectura hexagonal ofrece:

- **Comprobabilidad extrema:** Los casos de uso se pueden probar sin necesidad de levantar una base de datos real o un servidor web. Basta con inyectar adaptadores falsos (mocks) que simulen el comportamiento esperado. Esto acelera las pruebas unitarias y de integración.
- **Evolución incremental:** Permite añadir nuevos puertos y adaptadores a medida que el sistema crece, sin modificar el núcleo existente. Por ejemplo, si inicialmente solo se necesita una interfaz web, luego se puede agregar una API REST sin afectar el dominio.
- **Adaptabilidad a cambios tecnológicos:** Si un framework web queda obsoleto o se decide migrar de base de datos relacional a NoSQL, solo se deben reescribir los adaptadores correspondientes; el dominio permanece intacto.

Desventajas y críticas A pesar de sus beneficios, la arquitectura hexagonal no está exenta de inconvenientes:

- **Complejidad inicial:** Requiere una inversión significativa en el diseño de puertos e interfaces, lo que puede resultar excesivo para aplicaciones pequeñas o prototipos. Como señala Martín[21], *“la arquitectura hexagonal es una gran solución para sistemas complejos, pero no para scripts de una sola página”*.
- **Codificación indirecta:** A menudo se necesitan varias clases para implementar una funcionalidad simple (interfaz, adaptador, fábrica). Esto puede percibirse como *overengineering* si el dominio es trivial.
- **Curva de aprendizaje:** Los desarrolladores no familiarizados con los conceptos de puertos y adaptadores pueden tener dificultades para entender el flujo de la aplicación.

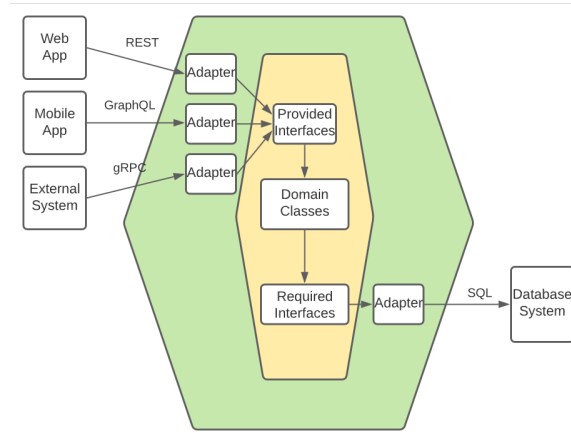


Figura 3.5: Esquema de la Arquitectura Hexagonal (Puertos y Adaptadores). Se observa el dominio en el centro, los puertos en los lados del hexágono y los adaptadores conectados a ellos. Imagen adaptada de Cockburn[19].

Patrón de Arquitectura Cebolla (Onion)

La Onion Architecture (o en capas de cebolla) fue implementada y presentada por Jeffrey Palermo en 2008. Como todo patrón de arquitectura, este hace hincapié en la división de funciones y el desarrollo de componentes interconectados. Desde que vio la luz en el año 2008 ha ganado popularidad entre los ingenieros de software que buscan crear aplicaciones escalables, con capacidad de mantenimiento y probadas.

Según el artículo 'Onion Architecture Used in Software Development' [22], publicado en ResearchGate (plataforma dedicada a la publicación de artículos científicos), la Onion Architecture organiza los componentes del software en capas concéntricas. Cada capa depende exclusivamente de la capa inmediatamente posterior. La lógica central de la aplicación se ubica en el centro de la arquitectura, rodeada por varias capas que la encierran y la protegen del exterior, razón por la cual este patrón se denomina "cebolla". La principal idea es que las aplicaciones sean divididas en cuatro capas:

1. **Dominio:** Esta capa actúa como el cerebro de la aplicación y representa los conceptos, entidades y reglas de negocio. No puede depender de otras capas, ya que es el centro de la arquitectura y tecnológicamente independiente.
2. **Infraestructura:** En esta capa se implementan las interfaces definidas por la capa de dominio. Además proporciona los elementos de infraestructura como mensajería, registro de eventos y acceso a datos.
3. **Aplicación:** La lógica de negocio relevante para la aplicación se encuentra en la capa de aplicación, que también actúa como enlace entre las capas de dominio

y presentación. Convierte los casos de uso y las reglas de negocio de la capa de dominio en acciones que la capa de presentación puede comprender.

4. **Presentación:** Esta capa ofrece la interfaz para que el usuario interactúe con la aplicación. Una aplicación web, una aplicación de escritorio o una aplicación móvil pueden conformar esta capa.

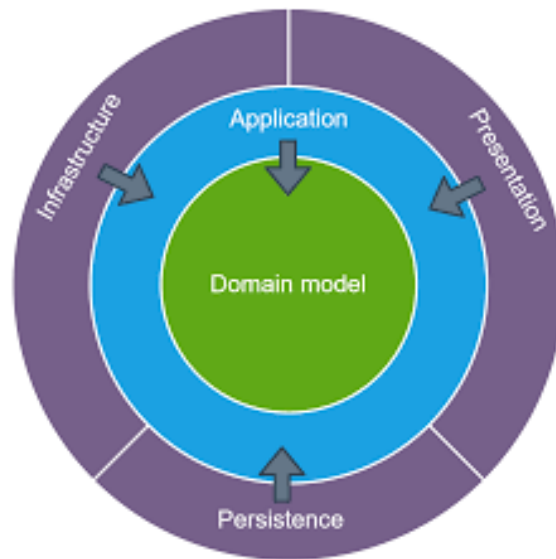


Figura 3.6: Esquema Onion Architecture, Tomado de Palermo (2008)

La figura 3.6 muestra un esquema de estructura de Onion Architecture, para comprender su diseño y el concepto de capas externas que “protegen” a la capa interna mencionado anteriormente. Esta arquitectura posee un gran número de ventajas y desventajas para el desarrollo de software, sin embargo para evitar la extensión del epígrafe se enunciarán unas pocas y de manera resumida.

Ventajas:

1. Separación de responsabilidades: La arquitectura de capas fomenta una clara división de responsabilidades entre las múltiples capas de la aplicación. Esto aumenta la modularidad de la aplicación y facilita su comprensión y mantenimiento.
2. Comprobabilidad: Debido a que las capas de la arquitectura de cebolla están conectadas de forma flexible y pueden probarse por separado, escribir pruebas automatizadas para la aplicación resulta más sencillo.
3. Escalabilidad: Gracias a que las capas de la arquitectura cebolla pueden escalarse independientemente unas de otras, las aplicaciones pueden escalarse horizontalmente. A medida que la aplicación se expande, esto ofrece mayor rendimiento y fiabilidad.
4. Capacidad de Mantenimiento: Los cambios en una capa no deberían afectar a las demás, ya que cada capa se encarga de una tarea específica. Esto simplifica la actualización y el mantenimiento de la aplicación a lo largo del tiempo.

Desventajas:

1. Mayor complejidad: Incluso en proyectos pequeños, la arquitectura cebolla puede dificultar el desarrollo de una aplicación. La aplicación puede volverse más difícil de comprender y modificar debido a los niveles e interfaces adicionales.
2. Curva de aprendizaje: La comprensión y aplicación adecuadas de la arquitectura pueden requerir más esfuerzo y formación para quienes no estén familiarizados con la Arquitectura Cebolla.
3. Sobrediseño: En proyectos pequeños, la arquitectura de cebolla puede, en ocasiones, estar sobrediseñada, lo que añade complejidad y sobrecarga.
4. Sobrecarga de rendimiento: La sobrecarga de rendimiento puede deberse a las capas e interfaces adicionales de la arquitectura de cebolla, especialmente en aplicaciones que requieren alto rendimiento o procesamiento en tiempo real.

Patrón de Arquitectura Limpia (Clean)

La Arquitectura Limpia (Clean Architecture) es un enfoque de diseño de software propuesto por Robert C. Martin que busca la independencia de los sistemas con respecto a frameworks, bases de datos y detalles de implementación[23]. A través de la separación de responsabilidades, Clean Architecture permite que los sistemas sean adaptables a cambios tecnológicos sin alterar la lógica de negocio principal.

Clean Architecture se basa en conceptos previos como el estilo por Capas, el patrón Hexagonal (Ports and Adapters) de Alistair Cockburn y la Arquitectura en Cebolla (Onion Architecture) de Jeffrey Palermo, pero con un enfoque más claro en la independencia de los componentes y la comprobabilidad del código. Clean Architecture se guía por cuatro principios fundamentales[24] los cuales serán explicados a continuación:

1. **Independencia de cualquier framework:** La arquitectura limpia debe poder aplicarse a cualquier sistema, independientemente del lenguaje de programación o las bibliotecas/frameworks que utilice. Las capas deben estar tan bien separadas que puedan sobrevivir individualmente, sin necesidad de elementos externos.
2. **Comprobabilidad:** Cada módulo, tanto UI, base de datos, conexión API Rest, etc., debe poder probarse individualmente.
3. **Interfaz de usuario (UI) independiente:** La interfaz de usuario debe poder cambiar sin interrumpir todo el sistema (Esta capa debería vivir de forma tan independiente como para ser desmontada y sustituida por otra).
4. **Base de datos independiente:** Al igual que en el punto anterior, esta capa debe ser tan modular como para agregar múltiples fuentes de datos e incluso múltiples fuentes del mismo tipo de datos.

Como se mencionó anteriormente, Clean Architecture toma fundamentos de los patrones Hexagonal y Onion, ambos explicados en este epígrafe. Por tanto también presenta una división por niveles. Existen bibliografías en las que el esquema para representar a este patrón es similar al esquema Onion, (figura 3.6) y en otras se representa mediante un cono, como se muestra en la figura 3.7

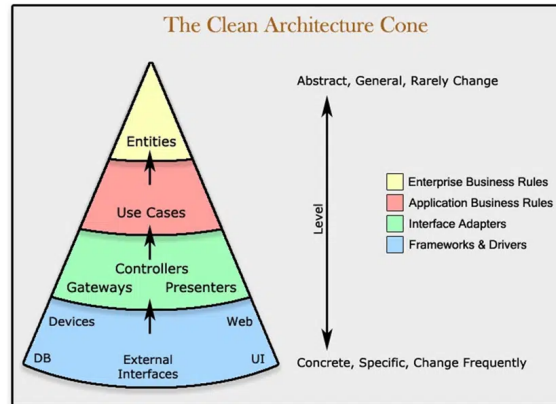


Figura 3.7: Esquema Clean Architecture en forma de Cono Adaptado de Martin (2012)

1. **Entidades:** Las entidades encapsulan las reglas comerciales de toda la empresa. Una entidad puede ser un objeto con métodos o puede ser un conjunto de funciones y estructuras de datos. No importa, siempre y cuando las entidades puedan ser utilizadas por muchas aplicaciones diferentes en la empresa.
2. **Casos de Uso:** El software de esta capa contiene reglas comerciales específicas de la aplicación. Encapsula e implementa todos los casos de uso del sistema. Estos casos de uso organizan el flujo de datos hacia y desde las entidades, y dirigen a esas entidades para que utilicen sus reglas de negocios en toda la empresa para lograr los objetivos del caso de uso.
3. **Adaptadores:** El software en esta capa es un conjunto de adaptadores que convierten datos del formato más conveniente para los casos de uso y entidades, al formato más conveniente para alguna agencia externa como la Base de Datos o la Web. De manera similar, los datos se convierten, en esta capa, de la forma más conveniente para las entidades y los casos de uso, a la forma más conveniente para cualquier marco de persistencia que se esté utilizando. Es decir, la base de datos. Ningún código dentro de este círculo debería saber nada sobre la base de datos. También en esta capa hay cualquier otro adaptador necesario para convertir datos de algún formato externo, como un servicio externo, al formato interno utilizado por los casos de uso y las entidades.
4. **Frameworks y controladores** La capa más externa generalmente se compone de frameworks y herramientas como la base de datos, el framework web, etc. Generalmente, no escribe mucho código en esta capa, aparte del código que se comunica con el siguiente círculo hacia adentro.

Arquitectura seleccionada para DermaUH 3.0

Una vez analizadas las alternativas arquitectónicas (Capas, Hexagonal, Onion y Clean), se optó por la arquitectura en capas como patrón de diseño para la aplicación web DermaUH 3.0. Esta decisión se fundamenta en cuatro criterios principales:

1. **Simplicidad y curva de aprendizaje:** La arquitectura en capas es ampliamente conocida y utilizada en la industria del software, lo que reduce la complejidad inicial del desarrollo y facilita la incorporación de nuevos desarrolladores al proyecto en el futuro[25].
2. **Experiencia previa del equipo:** El equipo de desarrollo cuenta con experiencia previa en la implementación de sistemas basados en capas, lo que acelera el desarrollo y minimiza riesgos técnicos.
3. **Rendimiento:** Patrones como Onion o Clean Architecture introducen una sobrecarga adicional debido a la inversión e inyección de dependencias, lo que puede afectar negativamente los tiempos de respuesta. Como se estableció en el requisito no funcional RNF3 (tiempo de respuesta inferior a dos segundos), la arquitectura por capas, al ser más directa en la comunicación entre capas, contribuye a cumplir con esta métrica[26]. Se deja como trabajo, hacer una verificación real del rendimiento.
4. **Flexibilidad para evoluciones futuras:** Aunque la arquitectura por capas puede tender al monolito, su organización permite extraer componentes específicos hacia microservicios con modificaciones mínimas. Por ejemplo, si en el futuro se decide externalizar el servicio de inteligencia artificial (modelo de clasificación de imágenes) como un microservicio independiente, la capa de negocio actual ya encapsula dicha lógica en componentes específicos (servicios de inferencia). De esta forma, solo sería necesario sustituir la llamada interna por una comunicación vía API REST, sin reescribir el resto del sistema[27].

Además, la arquitectura en capas es permisiva en cuanto a los patrones internos de cada capa. Esto permite adoptar incluso enfoques híbridos como por ejemplo, implementar el modelo ML en un servicio independiente (microservicio) y solamente modificar un poco la capa de negocio.

Como señalan Bass et al.[28], *“la arquitectura de software es el resultado de decisiones que equilibran restricciones técnicas, humanas y de negocio; no existe una solución única válida para todos los casos”*.

3.3.4. Modelado de la Base de Datos

Definida la arquitectura del sistema, el siguiente paso consiste en estructurar los datos que persistirán y las relaciones entre ellos. El presente apartado describe el modelo de datos de DermaUH 3.0, incluyendo las entidades principales (usuario, paciente, lesión, imagen, diagnóstico automático, entre otras), sus atributos, las claves primarias y foráneas, y las asociaciones que garantizan la integridad y coherencia de la información. Se presenta el Modelo Entidad-Relación, también conocido como Diagrama de Entidad Relación (MER).

Modelo de Entidad Relación (MER)

Según un artículo obtenido del sitio web UNIR[29], herramienta elaborada, desarrollada y diseñada con el propósito de ayudar a los estudiantes de universidades europeas, el modelo entidad relación (ERD o modelos ER) es una herramienta que permite representar de manera simplificada personas, objetos o conceptos que se relacionan entre sí. Se utiliza para exponer cómo se organiza la información en una base de datos. La creación del modelo entidad relación para su aplicación en el diseño de bases de datos se le atribuye a Peter Chen, profesor del MIT, quien publicó en 1976 el documento “Modelo entidad-relación: hacia una visión unificada de los datos”[30]. A continuación se expone el MER utilizado para la modelación de la base de datos. El diagrama fue realizado con la herramienta en línea Diagrams.net, plataforma gratuita con alta variedad de opciones para la realización de diagramas.

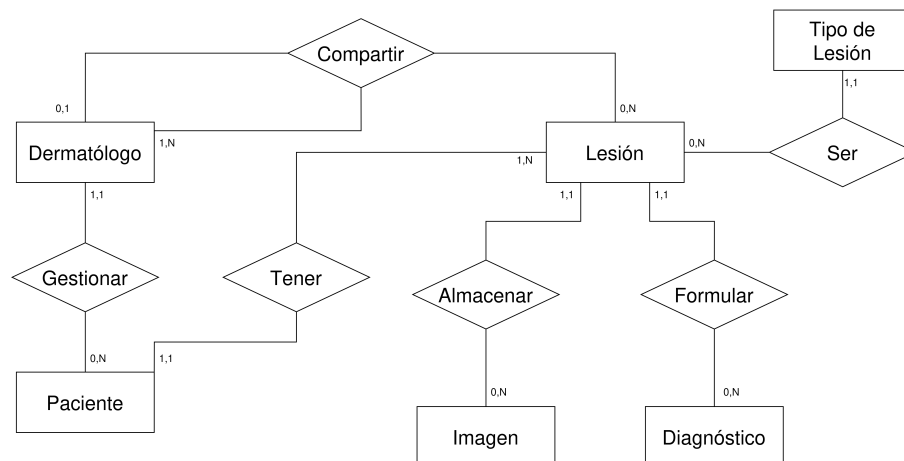


Figura 3.8: Modelo de Entidad Relacional MER

Paciente

El modelo de datos, **Paciente**, es la entidad más importante, ya que es el núcleo de una de las principales funcionalidades de DermaUH 3.0. A continuación una descripción de los atributos de dicha entidad.

- **idPaciente**: Llave primaria (PK), identificador único de tipo entero. Es generado automáticamente por el sistema de manera incremental.
- **name**, **lastName1**, **lastName2**: Campos para obtener el nombre completo del paciente. Todos son de tipo varchar, con una longitud máxima de cien caracteres por cada campo.
- **years**, **sex**, **skinColor**: Campos relacionados con la información étnica del paciente. Estos campos su principal funcionalidad es brindar más información para obtener un mejor diagnóstico asistido por parte de los modelos ML integrados en la nueva versión.
- **address**, **city**, **state**, **jobProfession**: Campos para reflejar la dirección y profesión laboral del paciente. Todos son de tipo varchar sin límites. Estas propiedades son importantes para saber el nivel exposición de la piel al sol, debido la heterogeneidad del clima en diferentes sitios del país.
- **registerData**: Campo de tipo Date para tener control de cuando fue que el paciente fue registrado por primera vez en el sistema de DermaUH.

Dermatólogo

Esta es una entidad fuerte, ya que no depende de otras entidades para instanciarse. El modelo **Dermatólogo**, es el responsable de representar la información de los usuarios del sistema. Dicho de otro modo, un dermatólogo es un usuario de DermaUH 3.0.

- **idDermatologo**: Llave primaria (PK), identificador único de tipo entero. Es generado automáticamente por el sistema de manera incremental.
- **name**, **lastName1**, **lastName2**: Campos para obtener el nombre completo del dermatólogo. Todos son de tipo varchar, con una longitud máxima de cien caracteres por cada campo.
- **email**: Campo de tipo text, se usa para guardar el email del dermatólogo. En el sistema este campo es utilizado como username (nombre de usuario) y como el identificador para hacer inicio de sesión.

- ci: Carnet de Identidad del Usuario, campo de tipo varchar.
- phoneNumber: Campo para guardar el número telefónico del usuario.
- hospital, state, city: Campos de tipo de varchar para guardar el hospital, municipio y provincia respectivamente del usuario.
- doctorId: Identificador que se genera según el hospital, municipio y provincia del dermatólogo.
- userType: Campo de tipo varchar. Solo hay tres posibles valores para este campo: Admin, Doctor, Resident, es utilizado para manejar los permisos y roles de los usuarios de la aplicación.
- tutorEmail: Campo de tipo varchar, almacena el correo del tutor en caso del usuario ser residente, o estudiante de intercambio. En caso contrario este campo se puede mantener null.

Lesión

Modela el comportamiento de las lesiones que puede presentar un paciente. Un paciente puede presentar una o muchas lesiones, por tanto el modelo **Lesión** depende de que exista un paciente. Se configuró con la regla de integridad referencial **ON DELETE CASCADE** (eliminación en cascada)[31], de modo que si un paciente es eliminado del sistema, todas sus lesiones asociadas se eliminan automáticamente.

- idLesion: Llave primaria (PK), identificador único de tipo entero. Es generado automáticamente por el sistema de manera incremental.
- date: Fecha en la que se registra la lesión. Es un campo de tipo date, no editable.
- phototype: Campo de tipo entero, solo puede tomar valores en el intervalo [1;6] que se usa para almacenar el fototipo² de la piel donde se encuentra ubicada la lesión. El fototipo no es más que la reacción de la piel ante la exposición a la luz solar.
- localization: Campo de tipo entero, toma valores en el intervalo [1;10] y almacena la parte del cuerpo donde se encuentra la lesión.
- histopathologicalDiagnosis: Campo de tipo bool. Indica si la lesión fue diagnosticada histopatológicamente, en otras palabras, afirma que se realizó biopsia y esta fue evaluada por un patólogo

²El fototipo cutáneo, según la escala de Fitzpatrick, clasifica la piel del tipo I (siempre se quema, nunca se broncea) al VI (nunca se quema, piel muy pigmentada).

- **injuryType**: Llave foránea (FK) que referencia a la entidad **Tipo de Lesión**. Es un campo de tipo entero.
- **patientId**: Llave foránea (FK) que referencia a la entidad **Paciente**. Es un campo de tipo entero.

Imagen

Modelo utilizado para guardar los metadatos[32] de las imágenes de las lesiones. Cada lesión puede tener muchas imágenes o no tener ninguna, al igual que la relación Paciente Lesión, esta también se configuró con la regla **ON DELETE CASCADE**.

- **idImagen**: Llave primaria (PK), identificador único de tipo entero. Es generado automáticamente por el sistema de manera incremental.
- **imagen**: Campo donde se almacenan los metadatos de la imagen en cuestión.
- **imagePath**: ruta del archivo de imagen en el servidor.
- **imageType**: Campo de tipo bool que indica True si una Imagen es dermatoscópica
- **commentary**: Campo de tipo texto, donde el usuario podrá exponer, si desea, una breve descripción de la imagen.
- **injuryId**: Llave foránea (FK) que referencia a la entidad **Lesión**

Diagnóstico

Este modelo se implementó para modelar los diagnósticos realizados por los doctores a lesiones registradas de pacientes. Una lesión puede tener varios diagnósticos, es válido tener diferentes criterios, por tanto la configuración de eliminación también es **Cascada**. A continuación los atributos de la entidad de Diagnóstico

- **idDiagnosis**: Llave primaria (PK), identificador único de tipo entero. Es generado automáticamente por el sistema de manera incremental.
- **date**: Fecha en la que se realiza el diagnóstico. Es un campo de tipo date, no editable.
- **commentary**: Campo de tipo de texto, no obligatorio, representa comentarios dejados por el dermatólogo respecto a su diagnóstico.
- **injuryId**: Llave foránea (FK) que referencia a la entidad **Lesión**

Capítulo 4

Detalles de Implementación

Para este momento, ya se tienen definidos los requerimientos funcionales y la arquitectura de software que mejor se adecúa a la aplicación DermaUH 3.0. En este capítulo se abordarán cuestiones relacionadas concretamente con la implementación de la aplicación.

4.1. Stack Tecnológico

Como se ha mencionado a lo largo de este documento, DermaUH 3.0 es una aplicación web, la cual tendrá integrado un modelo de Aprendizaje de Máquinas (ML por sus siglas en inglés). Por tal motivo se decidió optar por un conjunto de tecnologías ideales para el desarrollo web y que se puedan combinar sin problemas con técnicas de ML. En el capítulo anterior se seleccionó la arquitectura por capas como patrón principal diseño, dividiendo la aplicación en tres niveles: Presentación (UI, frontend), Lógica y Reglas de Negocio (backend) y Base de Datos (BD). Cada nivel tiene implementado con su propia tecnología, seleccionada luego de analizar los requerimientos de la aplicación, la experiencia del desarrollador y el tiempo disponible para la implementación.

React



Figura 4.1: Logo de React

Para la capa de presentación se ha seleccionado React, una biblioteca de JavaScript desarrollada por Meta para la construcción de interfaces de usuario basadas en componentes reactivos [33]. React ofrece ventajas determinantes para DermaUH 3.0, como el renderizado eficiente mediante el DOM virtual, que garantiza tiempos de respuesta rápidos incluso en dispositivos con recursos limitados, y la capacidad de crear una interfaz responsive que se adapta a resoluciones de 320px a 1920px, tal como exige el requisito no funcional RNF4. Su ecosistema permite integrar fácilmente gestores de estado como Redux o contextos nativos, lo que facilita la gestión de las sesiones de usuario autenticado con JWT y la visualización de historiales clínicos, lesiones y diagnósticos, adaptándose perfectamente a los casos de uso identificados para los actores Doctor y Administrador.

Django



Figura 4.2: Logo de Django

Para la capa de lógica de negocio se ha elegido Django, un framework de desarrollo web en Python que sigue el patrón MTV (Model-Template-View) y que incluye un potente sistema de rutas, autenticación y un ORM integrado [34]. Django proporciona ventajas críticas para DermaUH 3.0, como su módulo de autenticación que permite implementar de forma nativa el control de acceso basado en roles (administrador, doctor, residente) y la generación de JSON Web Tokens para cumplir con RNF1. Además, su arquitectura basada en aplicaciones reutilizables facilita la organización de los servicios de gestión de pacientes, lesiones, diagnósticos y la comunicación con el modelo de inteligencia artificial, permitiendo cumplir con los requisitos funcionales RF1 a RF5 y mantener un tiempo de respuesta inferior a dos segundos (RNF3) mediante el uso de vistas asíncronas y caché.

PostgreSQL



Figura 4.3: Logo de PostgreSQL

Para la capa de persistencia se ha optado por PostgreSQL, un sistema gestor de bases de datos relacional de código abierto reconocido por su solidez, cumplimiento de estándares SQL y soporte de tipado avanzado [35]. PostgreSQL se adapta a DermaUH 3.0 porque permite implementar exactamente el modelo entidad-relación definido en la figura 3.8, incluyendo claves foráneas con reglas ON DELETE CASCADE para mantener la integridad referencial entre pacientes, lesiones e imágenes, así como campos específicos como fototipos (enteros acotados) y rutas de archivos. Su soporte para conexiones cifradas mediante SSL/TLS garantiza el cumplimiento de RNF2, y su motor de consultas optimizado asegura que las operaciones de listado, búsqueda y filtrado de pacientes (con índices adecuados) respondan en los márgenes exigidos por RNF3 incluso con 50 usuarios concurrentes.

4.2. Implementaciones

En el apartado anterior, se definieron las tecnologías utilizadas para el desarrollo de la aplicación web DermaUH 3.0. En este apartado se explicará de cara de al desarrollador, la implementación concreta utilizada en cada capa, atendiendo al stack seleccionado en el apartado anterior. Se mostrarán además ejemplos de código y como se resolvieron “problemas” que fueron surgiendo durante el proceso de desarrollo.

4.2.1. Backend

Como se mencionó en el capítulo anterior, la arquitectura por capas posee un alto dinamismo, por tanto se pueden adoptar enfoques híbridos y cada nivel puede tener su propia implementación. Para organizar el backend, se optó por dividir la lógica en varias aplicaciones Django independientes (`auth_service`, `email_service`, `medical_service`, `image_service`). Cada una se encarga de un dominio específico, lo que facilita el mantenimiento y la evolución por separado. Aunque todas comparten la misma base de datos y se ejecutan en el mismo servidor (por lo que no se trata de una arquitectura de microservicios completa), esta modularidad permite, por ejemplo, reemplazar o actualizar el servicio de imágenes sin afectar al resto, siempre que se respeten las interfaces de comunicación. La Figura 4.4 muestra como fue dividido el backend en aplicaciones más pequeñas y de responsabilidad única. Se optó por este enfoque separatista por cuestiones de optimizaciones en las peticiones, ahora en vez de existir una sola aplicación que consuma todas las solicitudes, existen varias que leen y responden a solicitudes más concretas.

A continuación una breve explicación de cada servicio y que tareas cumple:

- **auth_service**: Las tareas de este servicio son las relacionadas con registro e inicio de sesión. Además maneja todas las operaciones CRUD sobre los usa-

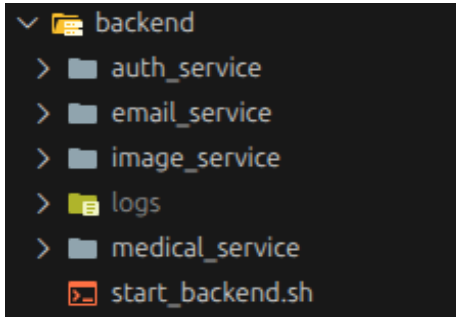


Figura 4.4: Servicios Django, la carpeta logs es para almacenar información de cada servicio

rios y es donde se aceptan o rechazan las solicitudes. También genera el JWT correspondiente una vez que un usuario inicie sesión en el sistema.

- **email_service**: Es donde se almacena el cliente de correo y donde se generan los mails que reciben los usuarios en relación a su solicitud.
- **medical_service**: Controla toda la gestión de pacientes, lesiones y diagnósticos. Es el servicio que procesa las solicitudes provenientes de usuarios con el rol de Doctor.
- **image_service**: Controla la parte de la aplicación dedicada a las imágenes: vincularlas a las lesiones y subirlas al sistema. En este servicio también se manejan las peticiones al modelo de ML vinculado a la aplicación.

Antes de comenzar el análisis detallado de cada servicio, es válido esclarecer conceptos que son muy habituales en el desarrollo web con Django como framework. Se incluirán también imágenes de código para un mejor entendimiento al lector. Toda la información se extrajo de la Documentación oficial de Django y del foro Django Rest-Framework¹.

Model

La clase Model de Django, es un objeto que se usa para modelar las entidades de la base de datos de un sistema. Todo Model debe estar vinculado con una única tabla. Además se deben regir por los siguientes fundamentos:

1. Cada modelo es una clase Python que hereda de la clase `django.db.models.Model`.

¹Django RestFramework es una extensión de Django muy utilizada para la construcción de Web APIs a base de Django

2. Cada atributo del modelo representa un campo de base de datos.
3. Con todo esto, Django ofrece una generación automática API de acceso a bases de datos.

La Figura 4.5 muestra un ejemplo de como definir una entidad en Django, concretamente la entidad dermatólogo. En la figura se muestra que la clase `Dermatologist` hereda de `AbstractUser` y `PermissionMixin`² ambas heredan de la clase `Model` de Django, por lo tanto `Dermatologist` hereda de `Model`.

```
from django.contrib.auth.models import AbstractBaseUser, PermissionsMixin
from django.db import models
from users.manager import DermatologistManager

class Dermatologist(AbstractBaseUser, PermissionsMixin):
    USER_TYPES = (
        ('doctor', 'Doctor'),
        ('resident', 'Resident'),
        ('admin', 'Admin')
    )
    #Required fields
    first_name = models.CharField(max_length=30, default= "Ramon")
    last_name1 = models.CharField(max_length=30, default= "Perez")
    last_name2 = models.CharField(max_length=30, default= "Perez")
    email = models.EmailField(unique=True)
    user_type = models.CharField(max_length=20, choices=USER_TYPES)

    #Doctor Fields
    dni = models.CharField(max_length=11, unique=True, null= True, blank=True)
    doctor_id = models.CharField(max_length= 30, blank=True, null=True)]

    #Resident Fields
    tutor_email = models.EmailField(null=True, blank=True)

    #Optional fields
    phone_number = models.CharField(max_length=20, blank=True, null=True)
    hospital = models.CharField(max_length=100, blank=True, null=True)
    state = models.CharField(max_length=100, blank=True, null=True)
    city = models.CharField(max_length=100, blank=True, null=True)

    is_active = models.BooleanField(default=False)
    is_staff = models.BooleanField(default=False)
    is_doctor = models.BooleanField(default=False)
```

Figura 4.5: Modelo `Dermatologist`, implementado en Django

Serializer

Los serializers (serializadores) en Django, funcionan como un traductor de información. Permiten que datos complejos como consultas e instancias de modelos, se conviertan en datos nativos de Python. La forma en que se utilizaron en el desarrollo de `DermaUH 3.0` fue para manejar las peticiones de tipo `POST`³, `PUT`⁴ y como

²Estas clases definen que el modelo en cuestión la aplicación debe tratarlo como usuarios y que puede tener uno o varios permisos respectivamente

³Peticiones de crear nuevas instancias de modelos para la BD

⁴Peticiones para actualizar valores de instancias guardadas en BD

DTO⁵. La Figura 4.6 evidencia el uso como DTO de un serializer, mientras que la Figura 4.7 muestra un ejemplo de un serializer usado para “atender” una solicitud de tipo POST.

```
class DermatologistDto(serializers.Serializer):
    register_id = serializers.IntegerField(source= 'register_id.id')
    full_name = serializers.CharField(max_length=255)
    email = serializers.EmailField()
    doctor_id = serializers.CharField(max_length=255)
    user_type = serializers.CharField(max_length=255)
```

Figura 4.6: Serializer utilizado como DTO

```
class RegisterDermatologistSerializer(serializers.ModelSerializer):
    password = serializers.CharField(write_only=True)
    confirm_password = serializers.CharField(write_only=True)
    class Meta:
        model = Dermatologist
        fields = [
            "first_name",
            "last_name1",
            "last_name2",
            "email",
            "dni",
            "doctor_id",
            "password",
            "confirm_password",
            "phone_number",
            "hospital",
            "state",
            "city"
        ]

    def validate(self, data):
        err = validate_password_strength(data["password"])
        if err:
            raise serializers.ValidationError(err)
        if data["password"] != data["confirm_password"]:
            raise serializers.ValidationError("Las contraseñas no coinciden.")
        if Dermatologist.objects.filter(email=data["email"]).exists():
            raise serializers.ValidationError("Email ya registrado")
        return data

    def create(self, validated_data):
        validated_data.pop("confirm_password")
        password = validated_data.pop("password")
        user = Dermatologist.objects.create(**validated_data, user_type='doctor', is_active=False)
        user.set_password(password)
        user.save()
        return user
```

Figura 4.7: Serializer para crear una solicitud de dermatólogo

Como se muestra además en la figura anterior, con el serializer se puede hacer validación de datos (función `validate`) y además se pueden crear y guardar instancias. Esto es muy ventajoso en el desarrollo de aplicaciones, los serializers tienen la capacidad tanto de traducir datos, validarlos y crear o actualizar instancias.

View

Las Views (vistas) en Django son el portal de la comunicación con aplicaciones externas, como el frontend. Su principal función es recibir las solicitudes y luego tomar la decisión de consultar un serializer o no. Generalmente las views que reciben

⁵Data Transfer Object, como su nombre indican, se utilizan para extraer información de la BD sin necesidad de devolver toda la instancia[36]

peticiones de tipo GET o DELETE, no requieren de un serializer, tiene sentido ya no tienen que actualizar ni escribir en BD. Django aporta la clase `APIView` que es la más utilizada en el desarrollo, sin embargo existen otras como `ListAPIView` o `DestroyAPIView` que también pueden usarse para listar o eliminar elementos respectivamente. La Figura 4.8 muestra una vista que maneja una petición POST, nótese como en el apartado “request” (cuerpo de la petición) se hace referencia al serializer mostrado en la Figura 4.7, evidenciando que el cuerpo de la solicitud en cuestión debe ser igual al de la clase `Meta` en el serializer

```
class RegisterView(APIView):
    permission_classes = [AllowAny]
    @extend_schema(
        request=RegisterDermatologistSerializer,
        responses=(201: dict),
        description="Registra un nuevo dermatólogo en el sistema."
    )
    def post(self, request):
        logger.info(f"Registro request received: {request.data}")
        try:
            serializer = RegisterDermatologistSerializer(data=request.data)
            if not serializer.is_valid():
                logger.error(f"Validation errors: {serializer.errors}")
                return Response(serializer.errors, status=400)
            user = serializer.save()
            logger.info(f"User created successfully: {user.email}")
            full_name = f"{user.first_name} {user.last_name1} {user.last_name2}"
            PendingDermatologist.objects.create(
                register_id= user,
                full_name= full_name,
                email= user.email,
                dni= user.dni,
                doctor_id= user.doctor_id,
                user_type= "Doctor"
            )
            EmailServiceClient.send_notification_new_user({
                "first_name": user.first_name,
                "last_name1": user.last_name1,
                "last_name2": user.last_name2,
                "email": user.email,
                "hospital": user.hospital
            })
            logger.info(f"Registration completed for: {user.email}")
            return Response({"message": "User created successfully"}, status=201)
        except Exception as e:
            logger.error(f"Unexpected error during registration: {str(e)}, exc_info=True")
            return Response({"detail": str(e)}, status=500)
```

Figura 4.8: View para peticiones Post

La Figura 4.9 muestra una view para peticiones GET, concretamente, el listado con filtro de los dermatólogos (usuarios) del sistema. En esta view se definen los diferentes tipos de filtros, se prepara el conjunto de datos a los cuales se les hará la consulta y finalmente se devuelven los usuarios correspondientes. Todo ese proceso lo realiza la clase `ListAPIView`, su funcionamiento puede ser consultado en la documentación oficial de RestFramework.

```
class ListDermatologistView(ListAPIView):
    permission_classes = [IsAdminUser]
    serializer_class = DermatologistDto

    filter_backends = [
        DjangoFilterBackend,
        SearchFilter,
        OrderingFilter
    ]

    # Filtros exactos
    filterset_fields = ["user_type"]

    # Búsqueda tipo LIKE
    search_fields = ["full_name", "email", "doctor_id"]

    # Ordenamiento
    ordering_fields = ["full_name", "email", "register_id"]

    def get_queryset(self):
        active = self.request.query_params.get("active")

        if active == "true":
            return ActiveDermatologist.objects.all()
        return PendingDermatologist.objects.all()

    @extend_schema(
        summary="Lista dermatólogos con filtro y paginación",
        description="""
        Filtros disponibles:
        - ?user_type=
        - ?search=texto
        - ?ordering=full_name
        - ?page=1
        """
    )

    def get(self, request, *args, **kwargs):
        return super().get(request, *args, **kwargs)
```

Figura 4.9: View para listado de usuarios

Una vez entendido los conceptos claves de la programación con Django, a continuación se hará una explicación más detallada de cada uno de los servicios que componen el backend de la aplicación.

Servicio de Autenticación (auth_service)

Como se mencionó al inicio de este apartado, las tareas del auth_service son aquellas relacionadas con la autenticación, registro y solicitudes de los usuarios del sistema, incluyendo la generación del JWT tras un inicio de sesión exitoso. Para garantizar la seguridad de las credenciales, las contraseñas se almacenan aplicando el algoritmo bcrypt con un factor de costo 12, nunca en texto plano. El JWT generado incluye los claims user_id, role (admin, doctor o residente) y un tiempo de expiración de 8 horas; las sesiones prolongadas se soportan mediante un token de refresco almacenado en una cookie HttpOnly, con renovación automática. El servicio también

valida las solicitudes de registro comprobando la unicidad del email y del carnet de identidad, persistiendo la solicitud en estado pendiente y notificando al administrador por correo. La Figura 4.10 muestra como se genera un token personalizado para poder establecer comunicación entre los distintos servicios.

```
class CustomTokenObtainPairSerializer(TokenObtainPairSerializer):
    @classmethod
    def get_token(cls, user):
        token = super().get_token(user)
        token['user_id'] = user.id
        token['user_name'] = f'{user.first_name} {user.last_name1} {user.last_name2}'
        token['is_authenticated'] = user.is_authenticated
        token['is_staff'] = user.is_staff
        token['is_doctor'] = user.is_doctor
        token['is_resident'] = user.is_resident
        token['is_active'] = user.is_active
        token['last_login'] = datetime.now()
        try:
            logger.debug(
                "Generated token for user=%s claims: user_id=%s is_staff=%s is_doctor=%s is_resident=%s is_active=%s",
                getattr(user, 'email', getattr(user, 'pk', '<unknown>')),
                token['user_id'], token['is_staff'], token['is_doctor'], token['is_resident'], token['is_active'],
            )
        except Exception:
            logger.exception("Error logging token claims")
        return token
```

Figura 4.10: Generación del token

Servicio de Correo Electrónico (email_service)

El servicio de correo electrónico se ocupa de la notificación asíncrona a los actores del sistema, sin requerir autenticación previa para recibir las peticiones. Este servicio gestiona tres tipos de envíos. En primer lugar, cuando un nuevo dermatólogo o residente completa el formulario de registro, el servicio notifica al administrador con los datos básicos del solicitante; si se trata de un residente, además se envía una notificación al tutor indicado para que conozca la petición. En segundo lugar, tras la revisión del administrador, el servicio informa al usuario si su solicitud ha sido aceptada o rechazada, adaptando el mensaje al resultado. Por último, cuando un usuario olvida su contraseña, el servicio genera y remite un enlace seguro para restablecerla. Todos los correos se entregan mediante la función `send_mail` de Django, y la configuración del remitente se obtiene de las variables de entorno, garantizando así la separación entre configuración y código.

Servicio Clínico (medical_service)

El servicio clínico constituye el núcleo funcional de la aplicación para los dermatólogos, ya que encapsula todas las operaciones relacionadas con la gestión de pacientes, lesiones y diagnósticos. A través de este servicio, el doctor autenticado puede registrar nuevos pacientes, consultar sus datos personales y de contacto, filtrar la lista de pacientes mediante búsquedas y eliminar aquellos que ya no estén bajo su seguimiento, dando cumplimiento al requisito funcional RF1. Sobre las lesiones, el servicio permite registrar su localización anatómica y fototipo cutáneo, y almacenar los diagnósticos

definitivos emitidos por el médico, manteniendo siempre la integridad referencial mediante las reglas de eliminación en cascada definidas en el modelo de datos. Además, implementa la funcionalidad de compartir lesiones con otros doctores, lo que permite que un especialista autorizado acceda a la información clínica de un paciente que no está directamente bajo su cargo, cumpliendo así el requisito RF2.

Servicio de Imágenes e Inteligencia Artificial (`image_service`)

El servicio de imágenes se especializa en la recepción, validación y procesamiento de las fotografías clínicas y dermatoscópicas enviadas por los dermatólogos. Una vez que el usuario autenticado sube una imagen asociada a una lesión, este servicio se encarga de almacenar los metadatos correspondientes (ruta del archivo, tipo de imagen, comentarios) en la base de datos y de gestionar el archivo físico en el servidor. Su funcionalidad más relevante consiste en actuar como puente hacia el modelo de inteligencia artificial: cuando el médico solicita una sugerencia de clasificación, el `image_service` envía la imagen al modelo previamente entrenado, recibe el resultado (tipo de lesión y nivel de confianza) y lo persiste como un diagnóstico automático pendiente de validación. De esta forma se satisface el requisito funcional RF3, garantizando que el especialista reciba una ayuda inmediata sin que el sistema reemplace su criterio clínico, y todo ello respetando el tiempo máximo de respuesta exigido en RNF3.

Comunicación entre Servicios

Cada servicio se comporta como una WebApi implementada con Django. Aunque no exista una dependencia directa entre estos, durante la etapa de desarrollo, surgieron inconvenientes que obligaron establecer de alguna forma comunicación entre los servicios. Un ejemplo de ello es la creación de un paciente, si se observa la descripción de esta entidad (Pág 37-38), se observa que tiene una llave foránea referenciando a la entidad Dermatólogo. Sin embargo, el servicio `medical_service` (donde se registran pacientes) no tiene conocimiento de la existencia de esta entidad. La solución está dada por el JWT generado en el `auth_service`, como se explicó anteriormente: el token generado guarda el claim del `user_id`, que es el identificador en base de datos del usuario actual. Por tanto cuando el usuario actual registre un paciente, el valor de la llave foránea a la entidad dermatólogo será ese `user_id` proveniente del token generado. La Figura 4.11 muestra como el `user_id` proveniente de la solicitud es enviado al serializer.

```
class PostPatientView(APIView):
    permission_classes = [IsDoctor]

    @extend_schema(
        tags=['Patient'],
        description='Create a new patient',
        request=PostPatientSerializer,
        responses={201: dict}
    )
    def post(self, request):
        serializer = PostPatientSerializer(
            data=request.data,
            context={'doctor_id': request.auth.get("user_id")}
        )
        serializer.is_valid(raise_exception=True)
        patient = serializer.save()
        return Response({
            "id": patient.id,
            "message": "Patient created successfully"
        }, status=201)
```

Figura 4.11: Uso de user_id proveniente del JWT

4.2.2. Frontend

La capa de presentación, fue desarrollada con el framework React. Para ello se optó por un patrón propio del framework: La Arquitectura basada en Características (Feature Architecture)[37] que consiste en dividir la aplicación frontend, en subaplicaciones más pequeñas, permitiendo una mayor capacidad de escalabilidad. Para el caso de DermaUH cada feature (característica) es una de las entidades principales del proyecto (Paciente, Dermatólogos, Lesiones). Este enfoque permite, al igual que en el backend, una separación de responsabilidades, donde cada feature se encarga de tareas únicas y no dependen una de la otra para poder ejecutarse. Con esta idea se logra que si por algún motivo, por ejemplo, el registro de pacientes deja de responder desde el backend, la aplicación continúe funcionando en el resto de los aspectos. Al observar la Figura 4.12 se puede apreciar la distribución de archivos de la arquitectura basada en características.

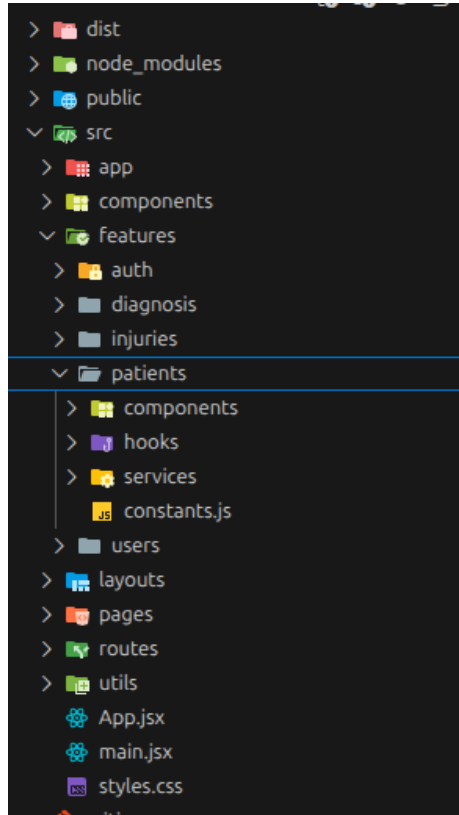


Figura 4.12: Arquitectura basada en características en el frontend DermaUH

Al igual que en el apartado Backend, se realizará un recorrido por diferentes conceptos que son importantes para entender las ideas detrás de la implementación del frontend con React.

Axios

Axios es una biblioteca de JavaScript que ayuda a la realización de peticiones HTTP desde la aplicación actual, hacia otros servidores o aplicaciones[38]. Se utiliza definiendo la dirección (URL) del sitio al cual se realizarán las peticiones. En caso de DermaUH, con backend en Django, este genera una URL para cada uno de los servicios⁶, entonces desde la capa de presentación, se define un axios para cada servicio del backend. La Figura 4.13 muestra como se usa axios para iniciar la comunicación con el `medical_service`. Haciendo uso del token guardado en el `localStorage`⁷ permite

⁶Recordar que se comportan como Web APIs independientes

⁷Es un diccionario global en el cual se pueden almacenar todo tipo de valores, en este caso se almacena el access token devuelto por el backend y se consume en el axios para enviar autenticación al resto de servicios

autenticarse en dicho servicio y utilizarlo si el usuario cuenta con los permisos correspondientes. Destacar también que axios establece comunicación de forma asíncrona mediante Promesas (Promise)[39].

```
1 import axios from "axios";
2 import { ACCESS_TOKEN } from "../../utils/constants";
3
4 export const apiMedical = axios.create({
5   baseURL: "http://localhost:8002/api/medical"
6 })
7
8 apiMedical.interceptors.request.use(
9   (config) => {
10     const token= localStorage.getItem(ACCESS_TOKEN)
11     if (token) {
12       config.headers.Authorization = `Bearer ${token}`
13     }
14     return config
15   },
16   (error) => {
17     return Promise.reject(error)
18   }
19 )
20
21 export default apiMedical
```

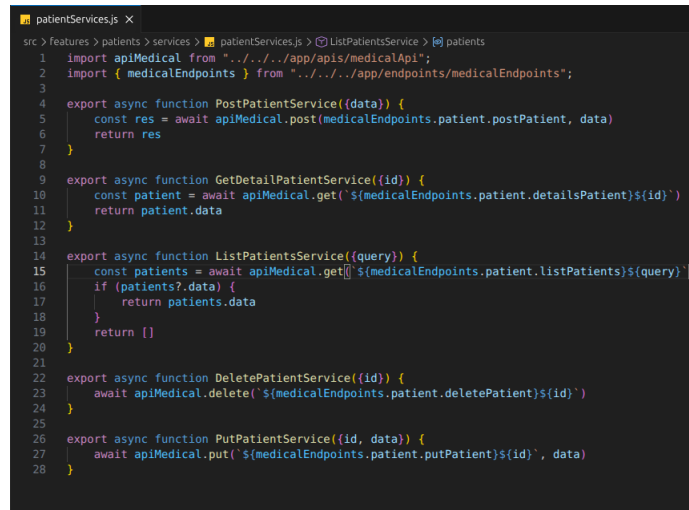
Figura 4.13: Configuración Axios para establecer comunicación

Services

Los Services (servicios) en React son funciones de corta extensión, normalmente no superior a las diez líneas de código, cuya responsabilidad exclusiva es realizar peticiones al backend utilizando el mensajero Axios previamente configurado. Actúan como una capa de abstracción que aísla la lógica de comunicación HTTP del resto de la aplicación, de modo que los componentes y hooks no necesitan conocer los detalles de las URLs, los métodos HTTP ni los encabezados de autenticación. En DermaUH 3.0 se ha organizado un servicio por cada entidad principal (Paciente, Lesión, Diagnóstico, Autenticación) y por cada servicio del backend (auth_service, medical_service, image_service, email_service). Cada servicio exporta funciones asíncronas que, utilizando la palabra clave **await**, invocan a Axios y retornan directamente la respuesta (generalmente en formato JSON) sin aplicar transformaciones adicionales, manteniendo así el principio de responsabilidad única.

La Figura 4.14 muestra como se configuran las funciones correspondientes a los servicios de la entidad Paciente, todas ellas añaden automáticamente el token JWT almacenado en **localStorage** a la cabecera **Authorization**, gracias a un interceptor configurado globalmente en Axios. Además, estos servicios incorporan un manejo básico de errores: si la petición falla (por problemas de red, autenticación o validación), lanzan una excepción que será capturada por el hook o componente que invocó al servicio, permitiendo mostrar mensajes claros al usuario como exige el requisito no funcional RNF5. De esta forma, los servicios actúan como el puente perfecto entre

la interfaz de usuario y la lógica de negocio remota, facilitando las pruebas unitarias (pueden ser mockeados fácilmente) y la evolución independiente de cada recurso.



```
src > features > patients > services > patientServices.js > ListPatientsService > patients
1 import apiMedical from "../../app/apis/medicalApi";
2 import { medicalEndpoints } from "../../app/endpoints/medicalEndpoints";
3
4 export async function PostPatientService(data) {
5   const res = await apiMedical.post(medicalEndpoints.patient.postPatient, data)
6   return res
7 }
8
9 export async function GetDetailPatientService({id}) {
10   const patient = await apiMedical.get(`${medicalEndpoints.patient.detailsPatient}${id}`)
11   return patient.data
12 }
13
14 export async function ListPatientsService({query}) {
15   const patients = await apiMedical.get(`${medicalEndpoints.patient.listPatients}${query}`)
16   if (patients?.data) {
17     return patients.data
18   }
19   return []
20 }
21
22 export async function DeletePatientService({id}) {
23   await apiMedical.delete(`${medicalEndpoints.patient.deletePatient}${id}`)
24 }
25
26 export async function PutPatientService({id, data}) {
27   await apiMedical.put(`${medicalEndpoints.patient.putPatient}${id}`, data)
28 }
```

Figura 4.14: Servicios de la entidad Paciente

Hooks

Los Hooks son funciones introducidas en React 16.8 que permiten a los componentes funcionales gestionar estado interno, efectos secundarios y contexto. Es una buena forma de encapsular los servicios definidos. La Figure 4.15 muestra como en un solo Hook se pueden configurar y guardar todos los servicios de la entidad paciente, como un diccionario en cualquier otro lenguaje de programación. Debido a su extensión solo muestran las funciones que invocan a los servicios de crear y listar pacientes.

Llama a la atención la función `useState`. Esta función también es un hook que se utiliza para manejar el estado local de los componentes o funciones. En la imagen se utiliza para asignar un valor inicial a las constantes `loading` y `error`, pero también se puede utilizar para formularios u objetos más complejos. En síntesis `useState` permite a los componentes funcionales tener su propio estado interno. Otro hook muy utilizado es `useEffect` fundamental en peticiones que requieren de forma necesaria un carácter asíncrono, como cargar listas o detalles de un paciente (o cualquier entidad).

```
export const usePatient = () => {
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState(null);

  const create = async ({data}) => {
    setLoading(true);
    setError(null);
    try {
      await PostPatientService({data});
    } catch (error) {
      setError(error.message);
      throw error;
    } finally {
      setLoading(false)
    }
  }

  const patients = async ({query}) => {
    setLoading(true);
    setError(null);
    try {
      const res = await ListPatientsService({query})
      console.log("Respuesta del Hook:", res)
      return res || []
    } catch (error) {
      setError(error.message)
      throw error
    } finally {
      setLoading(false)
    }
  }
}
```

Figura 4.15: Hook para los servicios de pacientes

Components

Los componentes son los bloques de construcción fundamentales de cualquier interfaz en React; cada componente es una pieza de UI independiente y reutilizable que puede recibir datos de entrada (props) y mantener su propio estado interno[40]. En DermaUH 3.0 se ha adoptado una arquitectura basada en características (Feature Architecture), lo que significa que los componentes se agrupan por dominio funcional: Paciente, Dermatólogo, Lesión y Diagnóstico. Por ejemplo, dentro de la característica Paciente existen componentes como **PatientList** (tabla o tarjeta de pacientes con botones de acción), **PatientForm** (formulario de registro y edición) y **PatientDetails** (vista detallada con pestañas para lesiones e historial). Cada uno de estos componentes se comunica con los servicios definidos en la capa inferior (los que utilizan axios) para solicitar o enviar datos al backend, y utilizan hooks para manejar el estado de carga, los errores y la respuesta. La separación en componentes pequeños y con una única responsabilidad facilita las pruebas unitarias, el mantenimiento y la evolución independiente de cada funcionalidad, de modo que si en el futuro se desea modificar el aspecto visual de la lista de pacientes, no es necesario alterar el formulario de registro

ni los componentes de lesiones.

4.3. Modelos Machine Learning

Como se ha hablado a lo largo de este documento, uno de los objetivos fundamentales de la versión 3.0 de DermaUH es la incorporación de nuevos modelos de procesamiento de imágenes entrenados con un conjunto de imágenes cubanas. Los modelos que se utilizaron fueron los desarrollados por el Licenciado en Ciencias de la Computación Daniel Abad, para más referencia puede consultar su tesis de diploma[41]. Se siguieron rigurosamente todas las instrucciones expuestas en el documento relacionada con las transformaciones del dataset en caso de ser necesarias.

El dataset utilizado tiene un total de 4576 imágenes. Fue dividido en cuatro clases Melanoma, Carcinoma Basal, Carcinoma Escamoso y Otras. La distribución por imágenes por categorías es la siguiente:

- Melanoma: 1000 para entrenamiento, 100 para pruebas y 100 para validación.
- Carcinoma Basal: 1000 para entrenamiento, 100 para pruebas y 100 para validación.
- Carcinoma Escamoso: 500 para entrenamiento, 64 para pruebas y 62 para validación.
- Otras lesiones: 1200 para entrenamiento, 200 para pruebas y 150 para validación.

Se expandió el conjunto de Carcinoma Escamoso para hacer que la cantidad de imágenes por categorías sea la misma y evitar sesgos. La técnica utilizada fue tomar la imagen original y su versión rotada 90 grados. Por tanto la cantidad de imágenes de Carcinoma Escamoso se duplicó, teniendo así 1000 imágenes de esta categoría. Si se analiza la tesis de Daniel[41] se explica que tener un dataset descompensado puede provocar overfitting y malos resultados, por ese motivo en esta tesis, no se hablará de experimentos con el dataset sin configuración.

El modelo utilizado es un híbrido que combina clasificación binaria (melanoma o no melanoma) y posteriormente clasificación multiclase (Carcinoma Basal, Carcinoma Escamoso, Otras lesiones). El análisis realizado en la tesis de Daniel[41] determina que esta combinación tiene una alta sensibilidad a la detección del melanoma, con un 99,7%, lo cual en su momento (versiones anteriores de DermaUH) fue un objetivo de vital importancia. Al entrenar este modelo con el dataset mencionado y modificado se obtuvieron las métricas expuestas en la tabla 4.1 y la matriz de confusión de la Figura 4.16.

Tabla 4.1: Métricas obtenidas con el modelo

Clase	Presición	Sensibilidad	F1
Melanoma	0.788	0.820	0.804
Carcinoma Basal	0.811	0.900	0.853
Carcinoma Escamoso	0.833	0.781	0.806
Otras lesiones	0.697	0.620	0.656

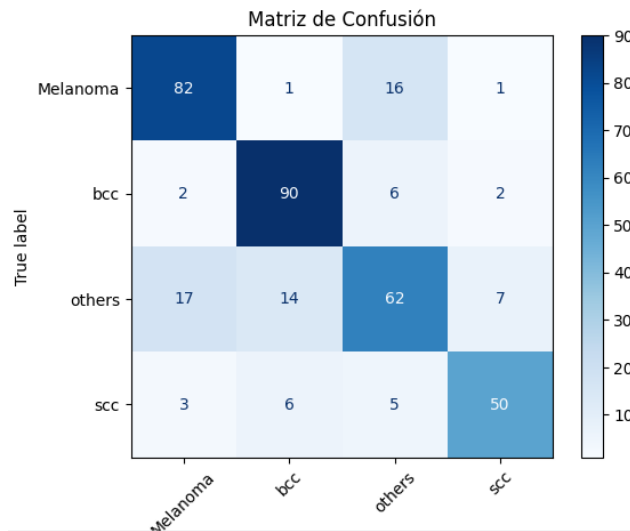


Figura 4.16: Matriz de confusión del modelo

Estos resultados se deben a la poca cantidad de imágenes del dataset, en la versión original del modelo se usó un dataset de mas de 50000 mil imágenes para entrenamiento y pruebas. Las métricas expresan una sensibilidad no baja al melanoma. Aunque sea menor que la anterior, es un buen número cuando se habla en el contexto médico, de cada 100 casos reales de melanoma, el modelo logra detectar 82. La categoría de otras lesiones es normal que se mantenga “baja”, ya que estas imágenes son muy heterógenas y no siguen siempre un patrón visual como puede ser el caso de los carcinomas. La matriz de confusión es la explicación perfecta de lo anterior, ya que muchas imágenes que deberían ser calificadas como “otras” son confundidas con melanomas y carcinomas basales.

Capítulo 5

Pruebas Funcionales

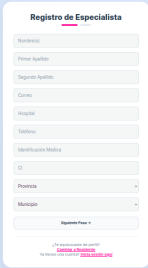
Las pruebas funcionales son esenciales en el desarrollo de software, ya que permiten mostrar y evidenciar el correcto funcionamiento de la aplicación. En este capítulo se realizarán distintas pruebas con el objetivo de verificar si todos los requisitos, casos de uso y objetivos de esta tesis, se lograron resolver.

5.1. Pruebas de Flujo Administrador

En este apartado se llevarán a cabo pruebas de flujo para comprobar el correcto funcionamiento de la aplicación, mostrando imágenes reales.

5.1.1. Solicitud de Registro

Con esta prueba se busca mostrar el correcto funcionamiento de la solicitud de registro, y luego su posterior activación. Para ello se cuenta con un usuario que tiene permisos de administración, es decir, como se mencionó al hablar de los diagramas UML, el actor Admin, es quien puede realizar las operaciones CRUD sobre los usuarios. La Figure 5.1, muestra como se ve un formulario de registro en DermaUH 3.0



Registro de Especialista

Nombre:

Primer Apellido:

Segundo Apellido:

Celular:

Hospital:

Teléfono:

Identificación Médica:

CI:

Profesión:

Municipio:

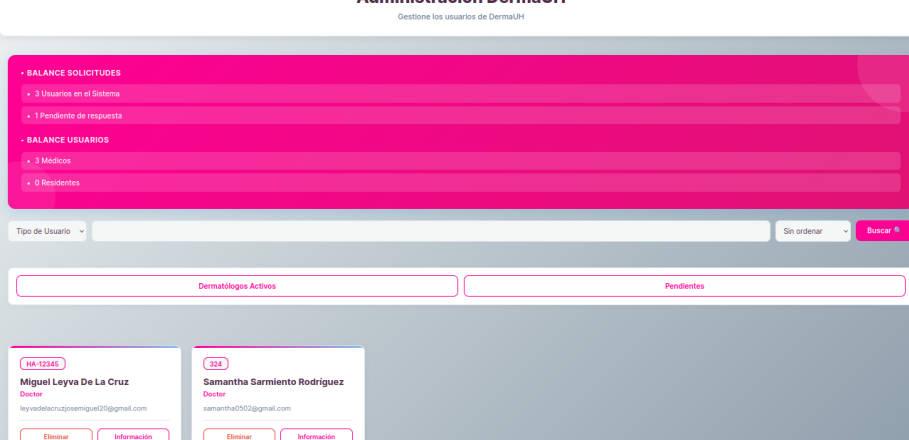
Registrar Perfil

¿Ya tienes una cuenta? [Regístrate aquí](#)

¿Ya tienes una cuenta? [Regístrate aquí](#)

Figura 5.1: Formulario de Registro

Una vez relleno el formulario, la solicitud es enviada al administrador, quien recibe un correo alertando que tiene un nuevo usuario pendiente de respuesta. Acto seguido el administrador entra al sistema con las credenciales correspondientes. La Figura 5.2 muestra como se ve la página principal de los usuarios, tiene un contador de solicitudes pendientes, doctores activos y residentes activos. Al presionar el botón “Pendientes”, se muestra la lista de usuarios que esperan confirmación para entrar al sistema, Figura 5.3.



Administración DermaUH
Gestione los usuarios de DermaUH

BALANCE SOLICITUDES

- 3 Usuarios en el Sistema
- 1 Pendiente de respuesta

BALANCE USUARIOS

- 3 Médicos
- 0 Residentes

Tipo de Usuario: Sin ordenar

HA-12345

Miguel Leyva De La Cruz

Doctor

leyvadelacruzjosemiguel20@gmail.com

324

Samantha Sarmiento Rodríguez

Doctor

samantha0202@gmail.com

Figura 5.2: Página del Administrador

Si se fija, en el listado de usuarios pendientes, cada tarjeta posee un botón “Activar”, al presionarlo se muestra el formulario de la Figure 5.4, el cual decide que permisos tendrá el usuario en cuestión. Solo se puede tener un permiso a la vez, el front se configuró de tal forma que solo se puede escoger uno.

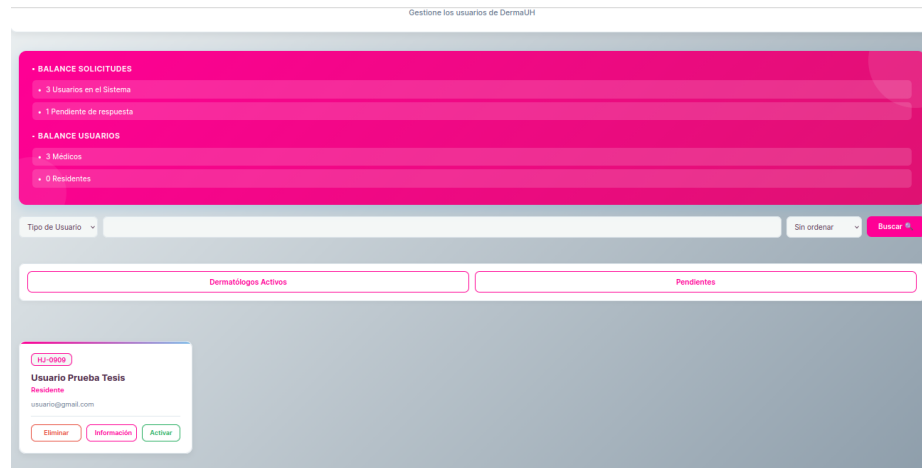


Figura 5.3: Usuarios Pendientes

The form is titled "Activa al Usuario". It contains three checkboxes for selecting the user's role:

- ☐ Es Doctor
- ☐ Es Residente
- ☐ Es Administrador

At the bottom of the form is a large pink button labeled "Activar Usuario".

Figura 5.4: Formulario Activar Usuario

Una vez escogido el rol del usuario, para este caso, se seleccionó doctor, se pulsa el botón activar. Si no hay ningún problema, debería aparecer un mensaje de usuario activado correctamente como se muestra en la Figure 5.5. Una vez activado el usuario, este recibe un correo electrónico vía gmail de que puede utilizar los servicios de DermaUH 3.0.

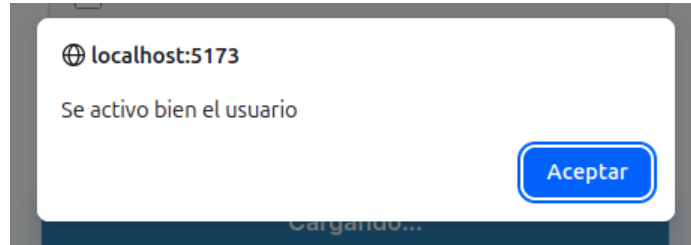


Figura 5.5: Mensaje de que el usuario fue activado con éxito

5.2. Pruebas de Flujo Doctor

Una vez analizadas las pruebas para el actor Admin, se procederá a realizar pruebas funcionales con un usuario cuyo rol es doctor. Las pruebas que se llevarán a cabo serán: CRUD sobre la entidad paciente, creación de una lesión, creación de un diagnóstico y subida de imagen, finalmente se compartirá la lesión con otro usuario, se entrará al sistema con sus credenciales y se verificará que la lesión fue compartida exitosamente.

5.2.1. CRUD sobre Paciente

Esta prueba se realiza con el objetivo de comprobar si la aplicación soluciona de forma correcta el RF1.

Conclusiones

Conclusiones

Recomendaciones

Recomendaciones

Bibliografía

- [1] R. S. J. y P. Norvig, *Artificial Intelligence: A Modern Approach*. Pearson, 2021 (vid. pág. 1).
- [2] T. M. Mitchell, *Machine Learning*. McGraw-Hill, 1997 (vid. pág. 2).
- [3] A. N. Hernández, «Inteligencia artificial explicativa en el análisis de imágenes médicas,» *Facultad Matemática y Computación, Universidad de La Habana*, 2025 (vid. págs. 5, 18).
- [4] DermEngine, *DermEngine App*, <https://www.dermengine.com/es/>, 2026 (vid. pág. 8).
- [5] VisualDx, *VisualDx App*, <https://www.visualdx.com/>, 2026 (vid. pág. 12).
- [6] I. Archive, *The International Skin Imaging Collaboration ISIC*, <https://www.isic-archive.com/>, 2026 (vid. pág. 14).
- [7] A. Yilmaz, S. Pekcan, Y. Gulsum y B. Temelkuran, «DERM12345: A Large, Multisource Dermatoscopic Skin Lesion Dataset with 40 Subclasses,» *Nature*, vol. I, 2024 (vid. pág. 17).
- [8] P. Tschandl, C. Rosendahl y H. Kittler, «The HAM10000 dataset: A large collection of multi-source dermatoscopic images of common pigmented skin lesions,» *Nature*, vol. V, n.º 180161, 2018 (vid. pág. 17).
- [9] M. Llamas-Velasco y A. García-Díez, «Cambio climático y piel: retos diagnósticos y terapéuticos,» *Actas Dermo-Sifilográficas*, vol. 101, n.º 5, 2010 (vid. pág. 17).
- [10] H. Cubero, «Procesos de ingeniería de software,» *Universidad de San Marcos*, vol. I, 2020 (vid. pág. 20).
- [11] Y. Luzhko, «¿Qué son las metodologías ágiles?» *Gerente de SEO*, vol. I, n.º 1, 2026. dirección: <https://payproglobal.com/es/respuestas/que-son-las-metodologias-agiles/> (vid. pág. 20).
- [12] C. Drumond, «¿Qué es scrum? Desglose del marco de trabajo ágil,» *Director de Marketing y Producto*, vol. I, n.º 1, 2025. dirección: <https://www.atlassian.com/es/agile/scrum> (vid. pág. 20).

- [13] M. A. Chaves, «La ingeniería de requerimientos y su importancia en el desarrollo de proyectos de software,» *InterSedes: Revista de las Sedes Regionales*, vol. VI, n.º 10, 2005 (vid. pág. 22).
- [14] M. C. O. Vidal, «Teoría Ingeniería de Software,» *Dpto. de Lenguajes y Sistemas Informáticos*, 2023 (vid. pág. 23).
- [15] M. M. Canelo, «Arquitectura de Software: Qué es y fundamentos clave,» *Marketing and Communications Manager en Profile*, vol. I, 2025 (vid. pág. 28).
- [16] O. Blancarte, «Arquitectura en Capas,» *Software Architect*, 2021 (vid. pág. 29).
- [17] D. Jackson, «Separation of concerns,» *Científico Computacional*, vol. I, n.º 1, 2013 (vid. pág. 30).
- [18] G. R. Garrido, «¿Qué es mock y fake en pruebas unitarias?» *Profile*, vol. I, n.º 1, 2025 (vid. pág. 31).
- [19] A. Cockburn, *Hexagonal Architecture*, <https://alistair.cockburn.us/hexagonal-architecture/>, Consultado: mayo 2026, 2005 (vid. págs. 31, 34).
- [20] A. Cockburn y J. M. G. de Paz, *Hexagonal Architecture Explained*, <https://www.hexagonalarchitecture.com/>, Consultado: mayo 2026, 2024 (vid. pág. 32).
- [21] R. C. M. (Bob), *The Clean Architecture and Ports & Adapters*, <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>, Consultado: mayo 2026, 2012 (vid. pág. 33).
- [22] S. M. A. Khan, «Onion Architecture Used in Software Development,» *National University of Computer and Emerging Sciences, Islamabad, Pakistan*, vol. I, n.º 1, 2023 (vid. pág. 34).
- [23] M. Tech, «Arquitectura Limpia (Clean Architecture),» *Blog de artículos relacionados con las ciencias computacionales*, vol. I, n.º 1, 2025 (vid. pág. 37).
- [24] J. Gonzales, «¿Qué es Clean Architecture y cuáles son sus beneficios y desventajas?» *Blog de Tecnología Ingeniería de Software y un poco más...*, vol. I, n.º 1, 2023 (vid. pág. 37).
- [25] M. Richards, *Software Architecture Patterns*, <https://www.oreilly.com/library/view/software-architecture-patterns/9781491971437/>, Capítulo 1: Layered Architecture, 2015 (vid. pág. 39).
- [26] M. Fowler, *Patterns of Enterprise Application Architecture*, <https://martinfowler.com/eaCatalog/>, Sección sobre “Layered Architecture and Performance”, 2002 (vid. pág. 39).

- [27] S. Newman, *Building Microservices*, <https://www.oreilly.com/library/view/building-microservices-2nd/9781492034018/>, Capítulo sobre From Monolith to Microservices, 2021 (vid. pág. 39).
- [28] L. Bass, P. Clements y R. Kazman, *Software Architecture in Practice*, 3rd. Addison-Wesley, 2012, ISBN: 978-0321815736 (vid. pág. 39).
- [29] T. R. Gonzales, «¿Qué es el modelo entidad relación y para qué se utiliza?» *UNIR*, vol. I, n.º 1, 2024 (vid. pág. 40).
- [30] P. Chen, *The Entity Relationship Model Toward a Unified View of Data*. 1976 (vid. pág. 40).
- [31] SupaBase, *Eliminaciones en cascada*, SupaBase, 2024 (vid. pág. 42).
- [32] J. de Andalucía, «¿Qué son los metadatos y para qué sirven?» *Consejería de Sostenibilidad y Medio Ambiente*, 2023 (vid. pág. 43).
- [33] I. Meta Platforms y R. Community, *React: A JavaScript library for building user interfaces*, <https://react.dev/>, Fecha de consulta: 2024-05-24, 2021 (vid. pág. 45).
- [34] D. S. Foundation, *Django: The Web framework for perfectionists with deadlines*, <https://www.djangoproject.com/>, Versión 4.1, 2022 (vid. pág. 45).
- [35] P. G. D. Group, *PostgreSQL: The World's Most Advanced Open Source Relational Database*, <https://www.postgresql.org/docs/>, Versión 16, lanzada en septiembre de 2023, 2023 (vid. pág. 46).
- [36] S. Logic, *Data Transfer Object DTO Definition and Usage*, <https://www.okta.com/identity-101/dto/>, 2024 (vid. pág. 49).
- [37] N. Rasouli, «Scalable React Projects with Feature-Based Architecture,» *Dev Community*, vol. I, n.º 1, 2025 (vid. pág. 54).
- [38] P. Halliday, *How To Use Axios with React*, <https://www.digitalocean.com/community/tutorials/react-axios-react>, 2025 (vid. pág. 55).
- [39] H. on React, *Promises and Async Await*, <https://handsonreact-com.translate.google/docs/promises-async-await/>, 2023 (vid. pág. 56).
- [40] R. Documentation, *Component*, <https://es.react.dev/reference/react/Component>, 2026 (vid. pág. 58).
- [41] D. Abad, «Propuesta de modelo de clasificación con alta sensibilidad al Melanoma,» *Facultad Matemática y Computación, Universidad de La Habana*, 2025 (vid. pág. 59).